



Basi di Dati

Progetto A.A. 2024/2025

# **SISTEMA PER LA GESTIONE DELLA VENDITA ALL'INGROSSO DI PIANTE**

Gabriele Monti

## **Indice**

|                                           |           |
|-------------------------------------------|-----------|
| <b>1. Descrizione del Minimondo .....</b> | <b>2</b>  |
| <b>2. Analisi dei Requisiti .....</b>     | <b>3</b>  |
| <b>3. Progettazione concettuale .....</b> | <b>9</b>  |
| <b>4. Progettazione logica .....</b>      | <b>17</b> |
| <b>5. Progettazione fisica .....</b>      | <b>28</b> |

## 1. Descrizione del Minimondo

### Sistema per la gestione della vendita all'ingrosso di piante

L'azienda Verde S.r.l. gestisce la vendita all'ingrosso di piante da interni ed esterni.

L'azienda tratta diverse specie di piante, ciascuna caratterizzata sia dal nome latino che dal nome comune, e da un codice univoco alfanumerico attraverso cui la specie viene identificata. Per ciascuna specie è inoltre noto se sia tipicamente da giardino o da appartamento e se sia una specie esotica o meno. Le piante possono essere verdi oppure fiorite. Nel caso di specie di piante fiorite, sono note tutte le colorazioni in cui una specie è disponibile.

L'azienda gestisce ordini massivi verso rivendite, ossia altre aziende. Per ogni rivendita sono noti la partita IVA, il nome e l'indirizzo della rivendita. È possibile mantenere un numero arbitrario di contatti, ad esempio numeri di telefono, di cellulare, di indirizzi e-mail. È necessario mantenere anche il nome/cognome di un referente. Le aziende devono avere un indirizzo di fatturazione, che può essere differente dall'indirizzo di spedizione dei pacchi.

I fornitori di Verde S.r.l. sono identificati attraverso un codice fornitore; per ciascun fornitore sono inoltre noti il nome, il codice fiscale ed un numero arbitrario di indirizzi. Il fornitore può fornire diverse specie di piante.

Verde S.r.l. ha un dipartimento di gestione di magazzino che tiene traccia delle giacenze ed effettua, periodicamente, ordini ai fornitori per mantenere una giacenza di tutte le specie di piante trattate.

Le specie di piante trattate sono gestite dai manager di Verde S.r.l.

Nell'ambito di un ordine è di interesse sapere a quale indirizzo questo deve essere inviato, e quale referente (se presente) e quale contatto fornire al corriere per mettersi in contatto con il destinatario in caso di problemi nella consegna. Non è possibile aprire un ordine se non vi è disponibilità in magazzino. L'ordine deve indicare quali specie di piante sono state ordinate ed in quali quantità.

Il listino prezzi, in cui si vuole tener traccia dei prezzi assunti nel tempo da ciascuna specie di piante.

Una variazione di prezzo non deve avere effetto su un ordine già aperto ma non ancora finalizzato. I prezzi sono gestiti dai manager di Verde S.r.l.

## 2. Analisi dei Requisiti

### Identificazione dei termini ambigui e correzioni possibili

| Linea      | Termine                               | Nuovo termine                        | Motivo correzione                                                                                                                                      |
|------------|---------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8          | Colorazioni                           | Colore dei fiori                     | Per precisare che la variazione indicata riguarda esclusivamente i fiori e non altre parti della pianta.                                               |
| 11, 25, 28 | Ordini                                | Ordini di vendita                    | Distinzione necessaria poiché il termine ordine si riferisce sia a quelli ricevuti dalle aziende rivenditrici e sia a quelli emessi verso i fornitori. |
| 11         | Rivendita                             | Azienda rivenditrice                 | Specifica che si tratta di aziende che acquistano piante all'ingrosso per rivenderle.                                                                  |
| 12         | Indirizzo della rivendita             | Indirizzo legale                     | Chiarisce che si tratta dell'indirizzo legale dell'azienda.                                                                                            |
| 20         | Dipartimento di gestione di magazzino | Magazzino                            | Semplificazione del termine.                                                                                                                           |
| 21         | Ordini                                | Ordini di rifornimento               | Distinzione necessaria per chiarire che si tratta degli ordini emessi dal responsabile logistico verso i fornitori.                                    |
| 23         | Manager                               | Responsabili commerciali e logistici | Distingue le aree di competenza di queste figure, ovvero la gestione logista e commerciale delle specie di piante.                                     |
| 26         | Contatto                              | Contatto telefonico                  | Specifica che il termine si riferisce ai recapiti telefonici.                                                                                          |
| 28         | Disponibilità                         | Giacenza                             | Giacenza è più preciso per indicare le quantità effettivamente disponibili in magazzino.                                                               |
| 31         | Listino prezzi                        | Storico dei prezzi                   | Termine più adatto per indicare che il sistema deve gestire una tracciabilità storica dei prezzi.                                                      |
| 34         | Finalizzato                           | Confermato                           | Confermato indica che l'ordine è stato verificato e accettato sia dal cliente che dal sistema.                                                         |
| 34         | Manager                               | Responsabili commerciali             | La gestione dei prezzi è riservata esclusivamente ai responsabili commerciali di Verde S.r.l.                                                          |

## Specifica disambiguata

Verde S.r.l. gestisce la vendita all'ingrosso di piante da interni ed esterni. L'azienda tratta diverse specie di piante, ciascuna caratterizzata dal nome latino, dal nome comune e da un codice univoco alfanumerico che identifica la specie. Per ciascuna specie è inoltre noto se sia tipicamente da giardino o da appartamento e se si tratti di una specie esotica o meno. Le piante possono essere verdi oppure fiorite. Nel caso di specie di piante fiorite, sono note tutte le colorazioni dei fiori in cui la specie è disponibile.

Verde S.r.l. gestisce ordini di vendita verso aziende rivenditrici. Per ogni azienda rivenditrice sono noti la partita IVA, il nome e l'indirizzo legale. È possibile mantenere un numero arbitrario di contatti (numeri di telefono, cellulare, indirizzi e-mail) e si registra il nome e il cognome di un referente. Le aziende devono inoltre avere un indirizzo di fatturazione, che può differire dall'indirizzo di spedizione dei pacchi.

I fornitori di Verde S.r.l. sono identificati tramite un codice fornitore. Per ciascun fornitore sono inoltre noti il nome, il codice fiscale e un numero arbitrario di indirizzi. Un fornitore può fornire diverse specie di piante.

Verde S.r.l. dispone di un magazzino che contiene fisicamente le giacenze delle specie di piante. Periodicamente il responsabile logistico effettua ordini di rifornimento verso i fornitori al fine di mantenere una giacenza adeguata di tutte le specie di piante trattate.

Le specie di piante sono monitorate dai responsabili logistici e commerciali di Verde S.r.l.

Nell'ambito di un ordine di vendita, è fondamentale sapere a quale indirizzo questo deve essere inviato, nonché individuare il referente (se presente) e il relativo recapito da fornire al corriere, per garantire una comunicazione immediata in caso di problemi nella consegna.

Non è possibile aprire un ordine di vendita se non vi è una giacenza sufficiente. L'ordine di vendita deve indicare quali specie di piante sono state ordinate e in quali quantità.

Lo storico dei prezzi tiene traccia dei prezzi assunti nel tempo da ciascuna specie di piante. Una variazione di prezzo non deve influire sugli ordini aperti, su quelli conclusi o su quelli confermati, ma si applica esclusivamente ai nuovi ordini successivi alla modifica. I prezzi sono gestiti dai responsabili commerciali di Verde S.r.l.

## Glossario dei Termini

| Termine                | Descrizione                                                                                                                       | Sinonimi                                       | Collegamenti                                                                            |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------|
| Colore dei fiori       | Le colorazioni disponibili per i fiori delle specie di piante fiorite.                                                            | Colorazioni                                    | Piante fiorite                                                                          |
| Ordini di vendita      | Ordini emessi dal responsabile commerciale per conto delle aziende rivenditrici.                                                  | Ordini (nel contesto della vendita)            | Aziende rivenditrici, ordini aperti, ordini conclusi, ordini confermati                 |
| Aziende rivenditrici   | Aziende che acquistano piante all'ingrosso da Verde S.r.l per la successiva rivendita.                                            | Rivendita, azienda                             | Ordini di vendita, indirizzo legale, indirizzo di fatturazione, indirizzo di spedizione |
| Indirizzo legale       | L'indirizzo di un'azienda usato per scopi amministrativi.                                                                         | Indirizzo della rivendita                      | Aziende rivenditrici, indirizzo di spedizione                                           |
| Contatti               | Recapiti associati alle aziende rivenditrici, comprendenti numeri di telefono, numeri di cellulare e indirizzi e-mail.            | Contatto                                       | Aziende rivenditrici, ordini di vendita                                                 |
| Fornitore              | Azienda o soggetto che fornisce le specie di piante a Verde S.r.l incluse nel proprio catalogo.                                   | Rivenditore                                    | Ordini di rifornimento, magazzino, specie di piante, responsabile logistico             |
| Magazzino              | Unità operativa di Verde S.r.l che contiene fisicamente le giacenze delle specie di piante.                                       | Dipartimento di gestione di magazzino          | Giacenze, ordini di rifornimento, specie di piante, responsabile logistico              |
| Giacenze               | Quantità effettivamente disponibili in magazzino per ciascuna specie di pianta.                                                   | Disponibilità, quantità disponibile            | Reparto logistico, specie di piante                                                     |
| Ordini di rifornimento | Ordini inviati ai fornitori dal responsabile logistico, per mantenere livelli adeguati di giacenza per ciascuna specie di pianta. | Ordini (nel contesto dei rifornimenti), scorta | Reparto logistico, fornitore, specie di piante                                          |

|                          |                                                                                                                                                                                       |                                  |                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------|
| Responsabili commerciali | Figure incaricate dell'inserimento e gestione delle specie di piante, del monitoraggio dello storico dei prezzi, della gestione degli ordini di vendita e delle aziende rivenditrici. | Manager, responsabile di settore | Storico dei prezzi, specie di piante, ordini di vendita, aziende rivenditrici |
| Responsabili logistici   | Figure incaricate della gestione operativa delle giacenze, dell'emissione degli ordini di rifornimento e della gestione dei rapporti con i fornitori.                                 | Manager, responsabile di settore | Giacenze, specie di piante, ordini di rifornimento, fornitori                 |
| Storico dei prezzi       | Registro che conserva la cronologia delle variazioni di prezzo associate a ciascuna specie di pianta.                                                                                 | Listino prezzi                   | Specie di piante, responsabile commerciale                                    |
| Ordini aperti            | Ordini di vendita o di rifornimento già registrati nel sistema, ma non ancora confermati. I prezzi non sono più modificabili.                                                         | Ordini in fase di elaborazione   | Ordini di vendita, ordini di rifornimento                                     |
| Ordini confermati        | Ordini approvati da Verde S.r.l. e dal cliente o fornitore, che non possono più essere modificati in alcuna delle loro componenti.                                                    | Ordini finalizzati               | Ordini di vendita, storico dei prezzi                                         |
| Ordini conclusi          | Ordini che hanno completato tutto il ciclo operativo, risultando spediti, ricevuti e chiusi.                                                                                          | Ordine concluso                  | Ordini di vendita, ordini di rifornimento                                     |

## Raggruppamento dei requisiti in insiemi omogenei

### Frasi relative alle specie di piante

L'azienda tratta diverse specie di piante, ciascuna caratterizzata dal nome latino, dal nome comune e da un codice univoco alfanumerico che identifica la specie.

Per ciascuna specie è inoltre noto se sia tipicamente da giardino o da appartamento e se sia una specie esotica o meno.

Le piante possono essere verdi oppure fiorite.

Nel caso di specie di piante fiorite, sono note tutte le colorazioni dei fiori in cui la specie è disponibile.

Le specie di piante gestite sono monitorate dai responsabili commerciali e logistici di Verde S.r.l.

### Frasi relative alla gestione degli ordini di vendita

Verde S.r.l. gestisce ordini di vendita verso aziende rivenditrici.

L'ordine di vendita deve indicare quali specie di piante sono state ordinate e in quali quantità.

Nell'ambito di un ordine di vendita, è fondamentale sapere a quale indirizzo questo deve essere inviato, nonché individuare il referente (se presente) e il relativo recapito da fornire al corriere, per garantire una comunicazione immediata in caso di problemi nella consegna.

Non è possibile aprire un ordine di vendita se non vi è una giacenza sufficiente in magazzino.

Una variazione di prezzo non deve influire sugli ordini di vendita aperti, su quelli conclusi o su quelli confermati, ma si applica esclusivamente ai nuovi ordini successivi alla modifica.

### Frasi relative alle aziende rivenditrici

Verde S.r.l. gestisce ordini di vendita verso aziende rivenditrici.

Per ogni azienda rivenditrice sono noti la partita IVA, il nome e l'indirizzo legale.

È possibile mantenere un numero arbitrario di contatti (numeri di telefono, cellulare, indirizzi e-mail) e si registra il nome e il cognome di un referente.

Le aziende devono inoltre avere un indirizzo di fatturazione, che può differire dall'indirizzo di spedizione dei pacchi.

### Frasi relative ai fornitori

I fornitori di Verde S.r.l. sono identificati tramite un codice fornitore.

Per ciascun fornitore sono inoltre noti il nome, il codice fiscale e un numero arbitrario di indirizzi.

Un fornitore può fornire diverse specie di piante.

**Frasi relative al magazzino e alle giacenze**

Verde S.r.l. dispone di un magazzino che contiene fisicamente le giacenze delle specie di piante.

Il responsabile logistico gestisce le giacenze e i rapporti con i fornitori.

Periodicamente il responsabile logistico effettua ordini di rifornimento verso i fornitori al fine di mantenere una giacenza adeguata di tutte le specie di piante trattate.

Non è possibile aprire un ordine di vendita se non vi è una giacenza sufficiente in magazzino.

**Frasi relative ai prezzi delle specie di piante e allo storico dei prezzi**

Lo storico dei prezzi tiene traccia dei prezzi assunti nel tempo da ciascuna specie di piante.

Una variazione di prezzo non deve influire sugli ordini aperti, su quelli conclusi o su quelli confermati, ma si applica esclusivamente ai nuovi ordini successivi alla modifica.

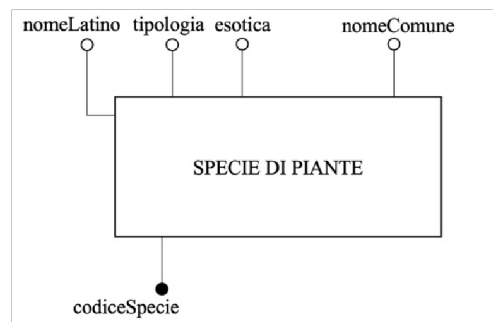
I prezzi sono gestiti dai responsabili commerciali di Verde S.r.l.

### 3. Progettazione concettuale

#### Costruzione dello schema E-R

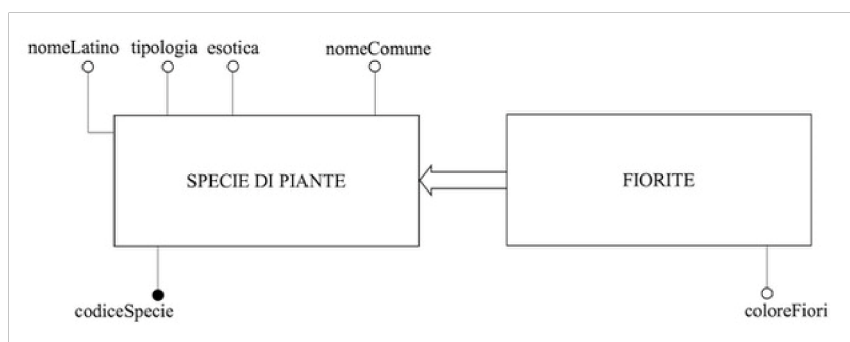
La progettazione concettuale è stata condotta seguendo un approccio di tipo inside-out, che si sviluppa a partire dall'individuazione dell'entità centrale del dominio applicativo, per poi estendersi progressivamente alle entità e alle relazioni collegate.

L'entità centrale individuata è *SPECIE DI PIANTE*, che costituisce il fulcro del sistema informativo.



Ciascuna specie è identificata univocamente da un codice alfanumerico, denominato *codiceSpecie*, ed è descritta attraverso attributi che permettono di registrare sia la denominazione comune *nomeComune*, sia quella scientifica *nomeLatino*.

Ad essa sono inoltre associati un attributo *tipologia*, che distingue le piante da giardino da quelle da appartamento, e un attributo booleano *esotica* che indica se la specie è autoctona o meno.

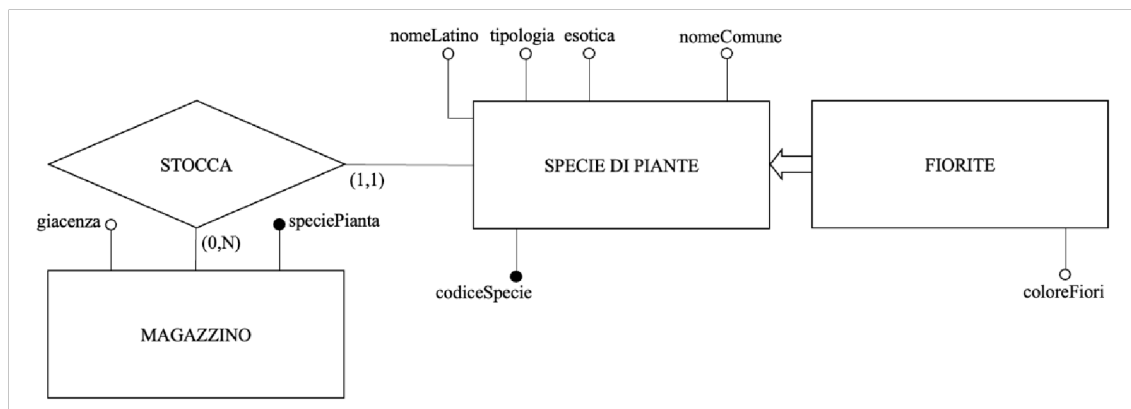


Per rappresentare le caratteristiche delle piante fiorite è stata introdotta una specializzazione parziale denominata *FIORITE*, che eredita gli attributi dell'entità padre *SPECIE DI PIANTE* e aggiunge l'informazione relativa al colore dei fiori, con l'attributo *coloreFiori*.

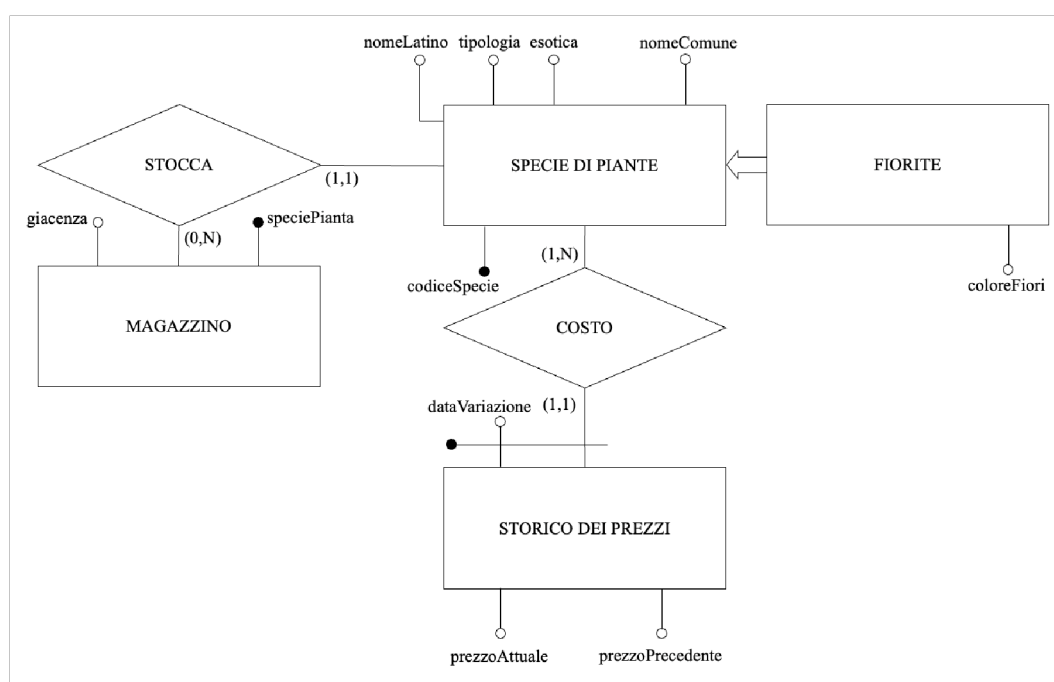
La gestione delle scorte fisiche è modellata attraverso l'entità *MAGAZZINO*, caratterizzata dall'attributo *giacenza* e dalla chiave primaria *speciePianta*.



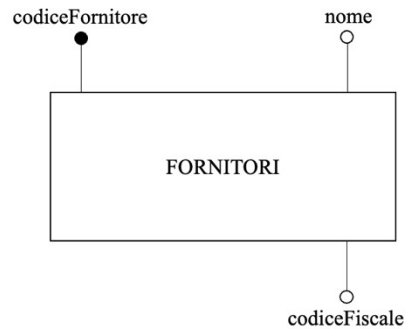
Poiché Verde S.r.l. dispone di un unico magazzino, esso è collegato all'entità *SPECIE DI PIANTE* dalla relazione *STOCCA*, la quale stabilisce che un magazzino possa contenere più specie, mentre ciascuna specie sia associata esclusivamente a un solo magazzino.



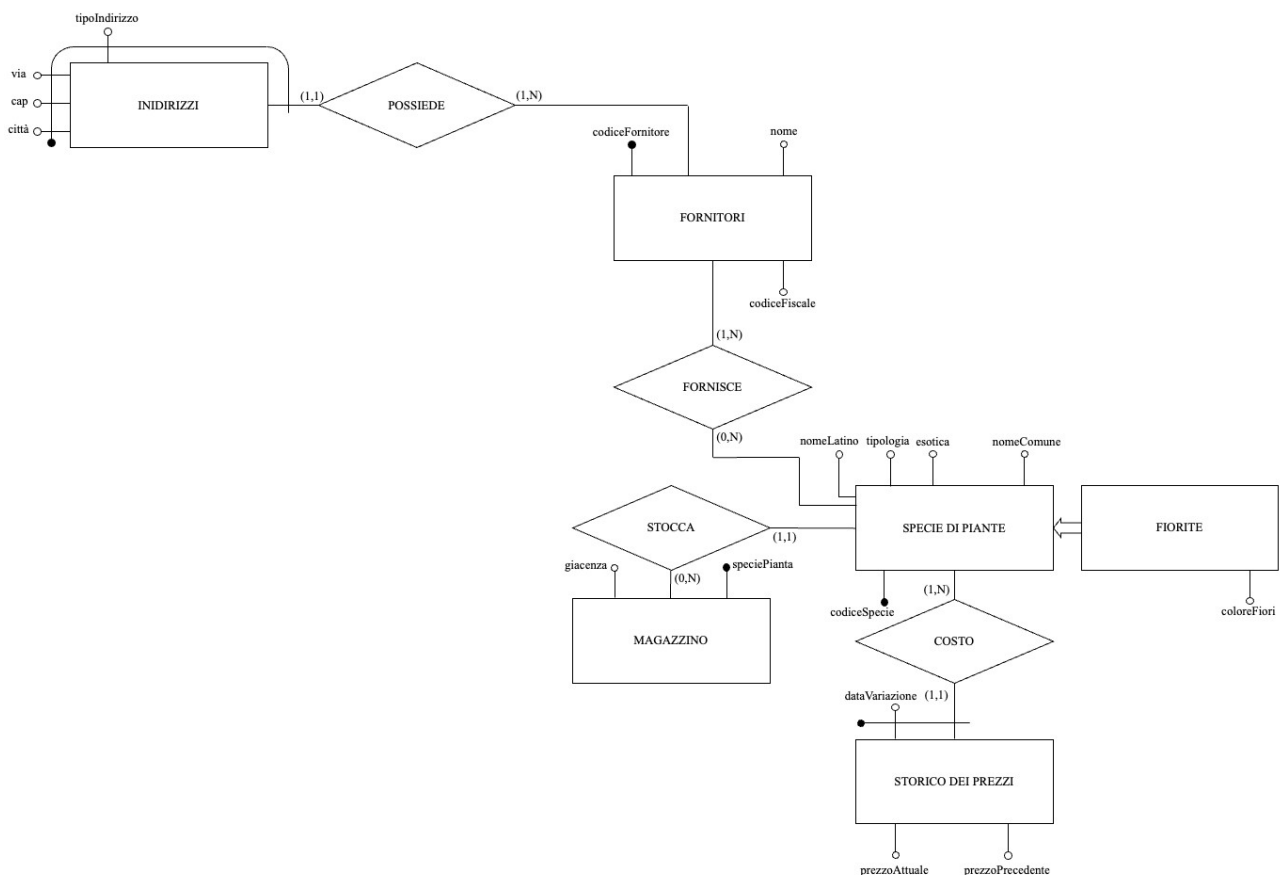
Per il monitoraggio dei prezzi è stata introdotta l'entità *STORICO DEI PREZZI*, collegata a *SPECIE DI PIANTE* tramite la relazione *COSTO*.



Lo storico consente di registrare il prezzo attuale e i prezzi precedenti, ciascuno corredato dalla data e dall'ora della variazione, permettendo così di tracciare con precisione l'evoluzione dei prezzi nel tempo. La memorizzazione dell'istante della variazione consente inoltre di registrare più modifiche di prezzo relative alla stessa specie anche all'interno della stessa giornata.

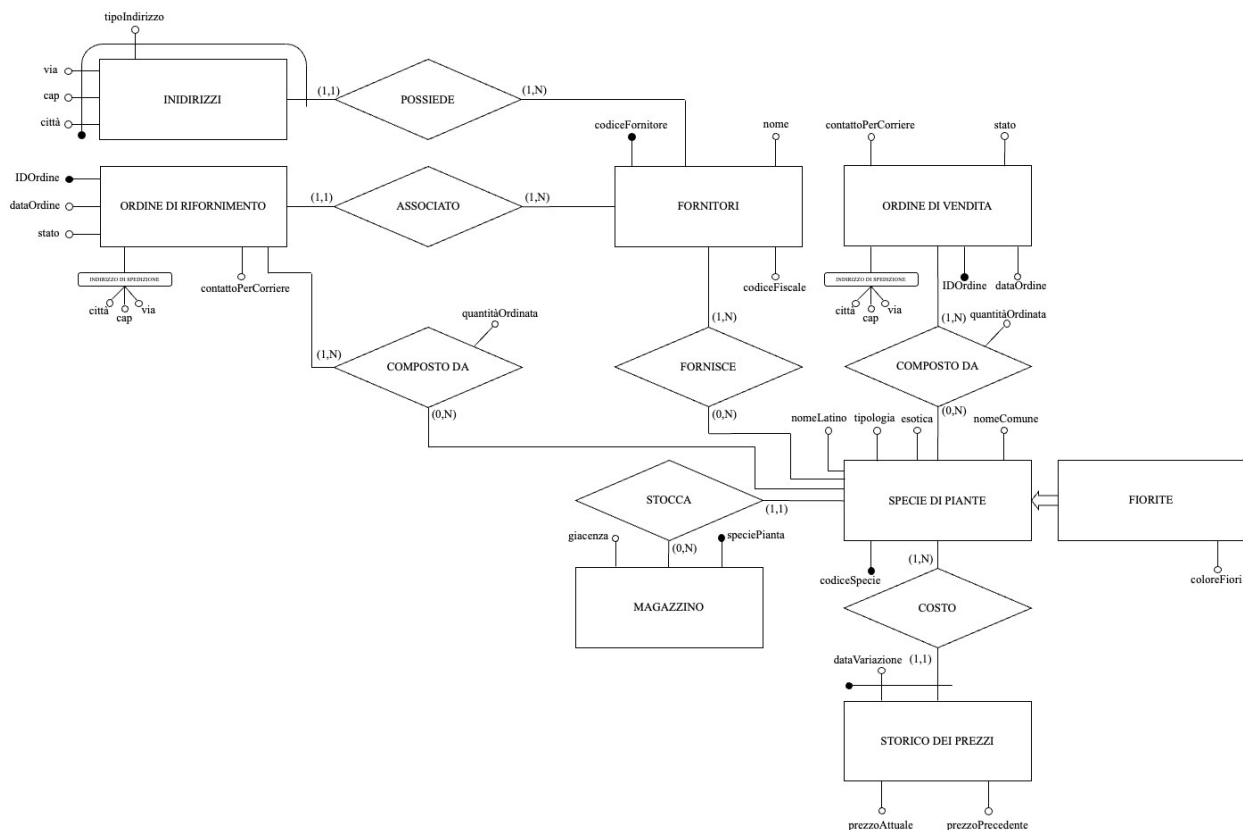


Le specie di piante sono fornite dai fornitori di Verde S.r.l modellati attraverso l'entità *FORNITORI*, identificati univocamente dal *codiceFornitore*. Per ciascun fornitore vengono memorizzati il *nome*, il *codiceFiscale* e un numero arbitrario di indirizzi di riferimento, legati all'entità *INDIRIZZI* attraverso la relazione *POSSIEDE*. L'indirizzo è composto da *via*, *città* e *cap*, e dal *tipoIndirizzo* che può assumere il valore legale o fatturazione.



La relazione *FORNISCE* esprime il legame multi-a-molti tra fornitori e specie di piante, in quanto un fornitore può fornire più specie e una specie può essere reperita presso fornitori diversi.

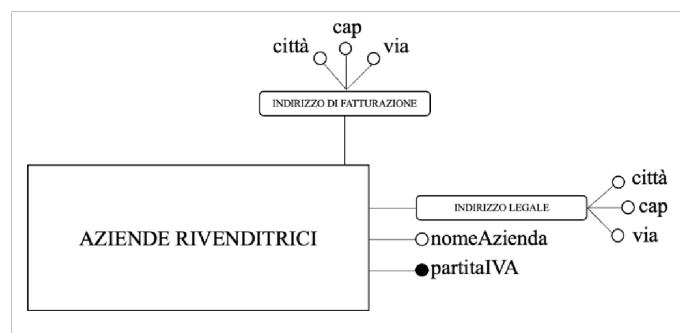
La gestione delle scorte è affidata all'entità *ORDINE DI RIFORMIMENTO*, che rappresenta gli ordini inviati dal responsabile logistico ai fornitori per rifornire le piante nel magazzino. Ogni ordine può contenere una o più specie di piante, con le relative *quantità*, descritte attraverso la relazione *COMPOSTO DA*.



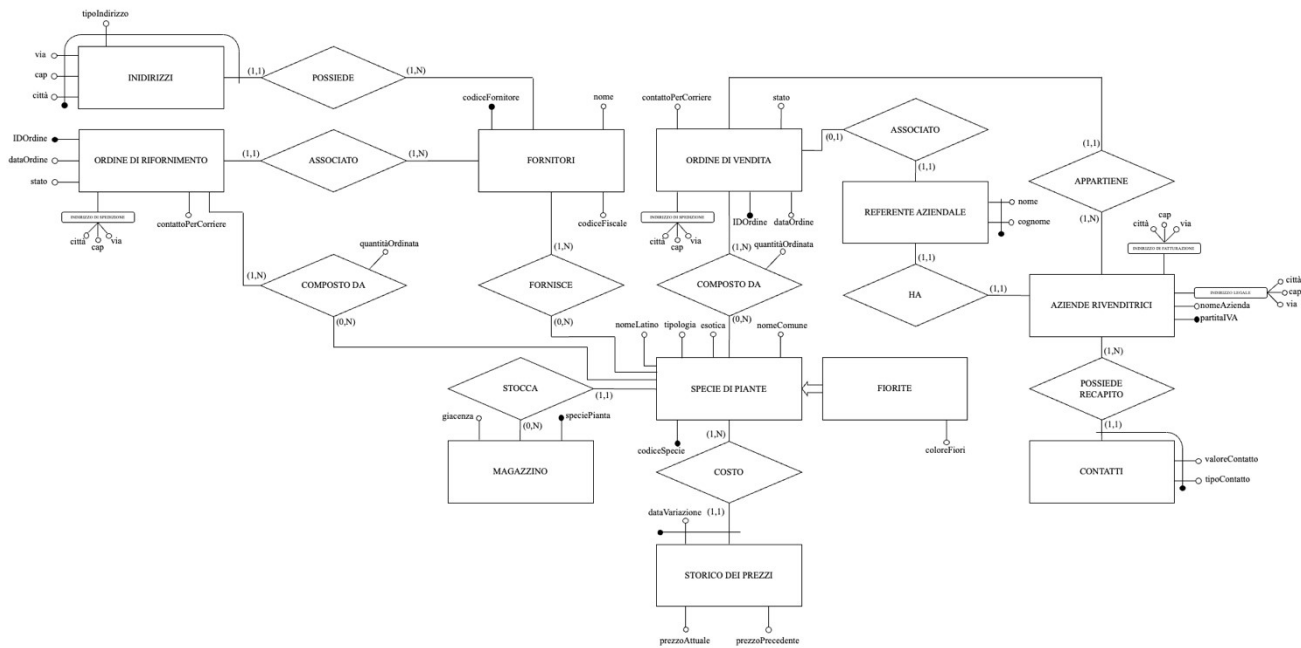
Sul fronte commerciale, il sistema prevede l'entità *ORDINE DI VENDITA*, identificata da *IDOrdine*, un codice numerico progressivo, caratterizzata da attributi quali *dataOrdine*, *stato* (aperto, confermato, spedito, consegnato, annullato) e il *contattoPerCorriere*. Inoltre, è presente un attributo composto che rappresenta l'indirizzo di spedizione dell'ordine, articolato nelle informazioni necessarie alla consegna.

Anche in questo caso la relazione *COMPOSTO DA* descrive la composizione delle righe dell'ordine, poiché un ordine può includere una o più specie di piante, ciascuna con la rispettiva *quantità* ordinata.

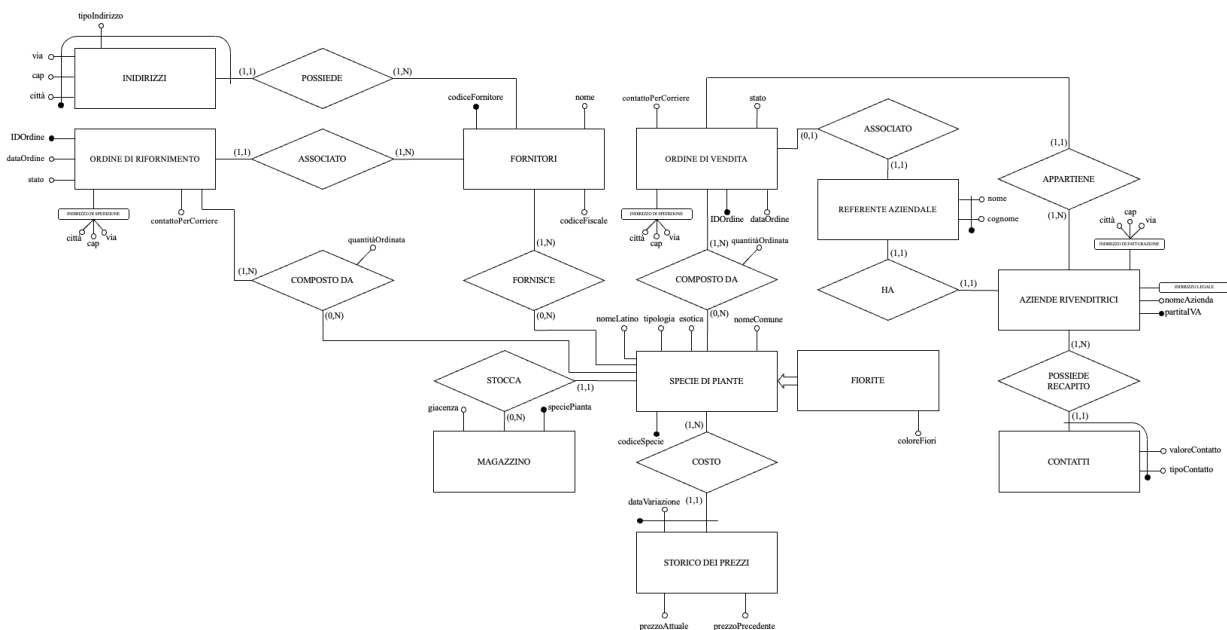
Ogni ordine di vendita è effettuato da un responsabile commerciale ed appartiene a un'azienda rivenditrice, identificata dalla *partitaIVA* e descritta da attributi quali *nomeAzienda*, *indirizzo legale*, *indirizzo di fatturazione* e *contatti multipli* (telefono, cellulare, e-mail) gestiti dall'entità *CONTATTI* legata ad *AZIENDE RIVENDITRICI* attraverso la relazione *POSSIEDE RECAPITO*.



Le aziende rivenditrici hanno un referente aziendale, rappresentato dall'entità *REFERENTE AZIENDALE*, la cui chiave primaria è composta dal *nome* e *cognome* del referente.



La gestione delle attività aziendali è affidata ai *RESPONSABILI DI SETTORE*, distinti in *RESPONSABILI COMMERCIALI* e *RESPONSABILI LOGISTICI*. Entrambi sono identificati da un *username* che funge da chiave primaria.



I responsabili commerciali si occupano della gestione delle specie di piante, del monitoraggio dei prezzi, dell'emissione e della gestione degli ordini di vendita, oltre che alla gestione delle aziende rivenditrici.

I responsabili logistici, invece, hanno la responsabilità della gestione delle giacenze, della gestione degli ordini di rifornimento e dei rapporti con i fornitori.

## **Regole Aziendali di Verde S.r.l.**

### **Regole descrittive (RD)**

**RD1:** Una specie di pianta è identificata da un codice univoco alfanumerico ed è caratterizzata da un nome latino, un nome comune, una tipologia (da appartamento o da giardino) e dalla natura (esotica o autoctona).

**RD2:** Una pianta può essere verde oppure fiorita. Nel caso delle piante fiorite, è noto il colore dei fiori disponibili.

**RD3:** Un'azienda rivenditrice è identificata dalla partita IVA, ha un nome e un indirizzo legale e può disporre di più contatti (telefono, cellulare, e-mail). Per ciascuna azienda rivenditrice è registrato un referente con nome e cognome. Ogni azienda rivenditrice ha un indirizzo di fatturazione, che può essere diverso dall'indirizzo di spedizione.

**RD4:** Un fornitore è identificato da un codice fornitore e da un codice fiscale, ha un nome e uno o più indirizzi. Ogni fornitore può fornire più specie di piante a Verde S.r.l.

**RD5:** Lo storico dei prezzi tiene traccia delle variazioni di prezzo di ciascuna specie di pianta nel tempo.

**RD6:** Verde S.r.l. dispone di un unico magazzino, che contiene le giacenze delle specie di piante gestite.

**RD7:** Gli ordini di vendita sono emessi dal responsabile commerciale di Verde S.r.l. per conto delle aziende rivenditrici e riportano le specie di piante ordinate insieme alle relative quantità e al prezzo unitario bloccato al momento dell'acquisto.

**RD8:** Gli ordini di rifornimento sono emessi dal responsabile logistico di Verde S.r.l. verso i fornitori al fine di mantenere una giacenza adeguata delle specie di piante trattate.

**RD9:** I responsabili commerciali gestiscono le specie di piante, le aziende rivenditrici, gli ordini di vendita e lo storico dei prezzi, mentre i responsabili logistici gestiscono le giacenze, i fornitori e gli ordini di rifornimento.

### **Regole di vincolo (RV)**

**RV1:** Non è possibile aprire un ordine di vendita se non vi è giacenza sufficiente delle specie richieste.

**RV2:** Ogni ordine di vendita deve indicare almeno una riga d'ordine, composta dalla specie di pianta ordinata e dalla relativa quantità.

**RV3:** Una variazione di prezzo non deve influire sugli ordini aperti, confermati e conclusi.

**RV4:** Le variazioni di prezzo si applicano esclusivamente ai nuovi ordini successivi alla modifica.

### Regole di derivazione (RDR)

**RDR1:** La giacenza disponibile di una specie coincide con la quantità memorizzata nel magazzino. Tale quantità viene aggiornata automaticamente in seguito alle operazioni che impegnano o rilasciano le specie negli ordini di vendita e in seguito alla consegna degli ordini di rifornimento.

**RDR2:** Il prezzo corrente di una specie si ottiene selezionando l'ultimo prezzo registrato nello storico dei prezzi per quella specie.

**RDR3:** Il totale di un ordine di vendita si ottiene sommando, per ciascuna riga d'ordine, il prodotto tra la quantità ordinata e il prezzo unitario memorizzato al momento dell'apertura dell'ordine. Il prezzo unitario associato alla riga rimane invariato per l'intero ciclo di vita dell'ordine, indipendentemente dalle successive variazioni registrate nello storico dei prezzi.

### Dizionario dei dati

| Entità                   | Descrizione                                                                      | Attributi                                                                                         | Identificatori                  |
|--------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------|
| Specie di piante         | Rappresenta le specie di piante gestite da Verde S.r.l.                          | Codice specie, nome latino, nome comune, tipologia (da giardino/da appartamento), esotica (sì/no) | Codice specie                   |
| Specie di piante fiorite | Specializzazione delle specie caratterizzate dalla presenza di fiori.            | Colore dei fiori (oltre agli attributi ereditati da specie di piante)                             | Codice specie                   |
| Aziende rivenditrici     | Aziende clienti che acquistano le piante da Verde S.r.l.                         | Partita IVA, nome azienda, indirizzo legale, indirizzo fatturazione                               | Partita IVA                     |
| Referente aziendale      | Persona di riferimento per ogni azienda.                                         | Nome, cognome                                                                                     | Nome + cognome                  |
| Contatti                 | Recapiti associati alle aziende rivenditrici.                                    | Tipo contatto (telefono, cellulare, email), valore contatto                                       | Tipo contatto + valore contatto |
| Ordini di vendita        | Ordini emessi dal responsabile commerciale per conto delle aziende rivenditrici. | Id ordine, data ordine, stato, indirizzo spedizione, contatto per il corriere                     | Id ordine                       |

|                                  |                                                                          |                                                                   |                                        |
|----------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------|
| Dettaglio ordine di vendita      | Specifica le piante e le quantità ordinate in ciascun ordine di vendita. | Quantità ordinata                                                 | Id ordine + codice specie              |
| Fornitori                        | Aziende o che forniscono le piante a Verde S.r.l.                        | Codice fornitore, nome, codice fiscale                            | Codice fornitore                       |
| Indirizzi fornitori              | Indirizzi associati ai fornitori.                                        | Tipo indirizzo, via, città, cap                                   | Codice fornitore + tipo indirizzo      |
| Ordini di rifornimento           | Ordini effettuati dal responsabile logistico verso i fornitori.          | Id ordine, data ordine, stato ordine                              | Id ordine                              |
| Dettaglio ordine di rifornimento | Specifica le specie e le quantità richieste nei rifornimenti.            | Quantità ordinata                                                 | Id ordine rifornimento + codice specie |
| Indirizzi                        | Rappresenta gli indirizzi associati ad aziende rivenditrici e fornitori. | Id indirizzo, tipo, via, città, cap                               | Id indirizzo                           |
| Magazzino                        | Struttura fisica dove sono stoccate le specie di piante.                 | Specie di pianta, giacenza                                        | Specie di pianta                       |
| Storico dei prezzi               | Registro delle variazioni di prezzo di ciascuna specie.                  | Codice specie, data variazione, prezzo precedente, prezzo attuale | Codice specie + data variazione        |
| Responsabili di settore          | Persone che gestiscono Verde S.r.l.                                      | Username, ruolo                                                   | Username                               |

## 4. Progettazione logica

### Volume dei dati

#### Stima Entità

| Concetto nello schema          | Tipo <sup>1</sup> | Volume atteso | Note                                                |
|--------------------------------|-------------------|---------------|-----------------------------------------------------|
| Specie di piante               | E                 | ~ 500         |                                                     |
| • di cui fiorite               | E                 | ~ 250         |                                                     |
| • di cui verdi                 | E                 | ~ 250         |                                                     |
| Aziende rivenditrici           | E                 | ~ 200         |                                                     |
| Fornitori                      | E                 | ~ 50          | Ognuno fornisce in media 30 specie                  |
| Ordini di vendita              | E                 | ~ 4000        | Media di 20 ordini annuali per azienda rivenditrice |
| Ordini di rifornimento         | E                 | ~ 400         | Circa 6-8 a settimana                               |
| Storico dei prezzi             | E                 | ~ 1500        | 3 variazioni annue × 500 specie                     |
| Giacenze                       | E                 | ~ 500         |                                                     |
| Contatti                       | E                 | ~ 600         | Media di 3 recapiti × 200 aziende                   |
| Referenti aziendali            | E                 | ~ 200         | Uno per azienda rivenditrice                        |
| Indirizzi aziende rivenditrici | E                 | ~ 400         | 200 fatturazione + 200 legali                       |
| Indirizzi fornitori            | E                 | ~ 100         | 50 fornitori × 2 indirizzi                          |
| Responsabili commerciali       | E                 | ~ 10          |                                                     |
| Responsabili logistici         | E                 | ~ 5           |                                                     |

#### Stima delle Relazioni

| Concetto nello schema               | Tipo <sup>1</sup> | Volume atteso | Note                                  |
|-------------------------------------|-------------------|---------------|---------------------------------------|
| COMPOSTO DA (ordini vendita)        | R                 | ~ 28000       | 4000 ordini × 7 specie medie a ordine |
| COMPOSTO DA (ordini rifornimento)   | R                 | ~ 4000        | 400 ordini × 10 specie medie a ordine |
| FORNISCE (fornitore-specie)         | R                 | ~ 1500        | 50 fornitori × 30 specie medie        |
| STOCCA (magazzino-specie)           | R                 | ~ 500         | 1 magazzino × 500 specie              |
| COSTO (specie-storico prezzi)       | R                 | ~ 1500        | Una riga per variazione di prezzo     |
| POSSIEDE (fornitoreindirizzo)       | R                 | ~ 100         | 50 fornitori × 2 indirizzi            |
| POSSEDE RECAPITO (azienda-contatti) | R                 | ~ 600         | 200 aziende × 3 contatti              |

<sup>1</sup> Indicare con E le entità, con R le relazioni

### Tavola delle operazioni

La seguente tabella riporta le operazioni non banali che il sistema informativo deve supportare, con la relativa frequenza stimata su base annua. Le stime sono state ricavate dall'analisi dei requisiti e dal volume operativo previsto per Verde S.r.l.

| Cod. | Descrizione                                                                                                 | Frequenza attesa |
|------|-------------------------------------------------------------------------------------------------------------|------------------|
| I1   | Inserisci nuova specie di pianta nel catalogo                                                               | ~ 50/anno        |
| I2   | Inserisci nuova azienda rivenditrice                                                                        | ~ 30/anno        |
| I3   | Inserisci nuovo fornitore                                                                                   | ~ 5/anno         |
| I4   | Inserisci nuovo ordine di vendita                                                                           | ~ 4000/anno      |
| I5   | Inserisci dettagli ordine di vendita                                                                        | ~ 28000/anno     |
| I6   | Inserisci nuovo ordine di rifornimento                                                                      | ~ 400/anno       |
| I7   | Inserisci dettaglio ordine di rifornimento                                                                  | ~ 4000/anno      |
| I8   | Inserisci variazione di prezzo                                                                              | ~ 1500/anno      |
| U1   | Aggiorna giacenza del magazzino dopo ogni ordine di vendita aperto e ogni ordine di rifornimento consegnato | ~ 32000/anno     |
| U2   | Aggiorna stato ordine di vendita*                                                                           | ~ 12000/anno     |
| U3   | Aggiorna stato ordine di rifornimento                                                                       | ~ 1200/anno      |
| R1   | Visualizza l'intero catalogo con prezzi e giacenze                                                          | ~ 5000/anno      |
| R2   | Visualizza storico prezzi                                                                                   | ~ 1500/anno      |

\***Aggiorna stato ordine:** quando un ordine viene creato lo stato iniziale è impostato su APERTO, successivamente può subire questi aggiornamenti:

- CONFERMATO – SPEDITO - CONSEGNA TO
- CONFERMATO - ANNULLATO
- APERTO - ANNULLATO

## Costo delle operazioni

### I1 Inserisci nuova specie di pianta nel catalogo

| Entità                  | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|-------------------------|-----------------|---------|-----------|------------------|-------------|
| CodiceSpecieProgressivo | 50              | 1       | 2         | 5                | 250         |
| Specie di pianta        | 50              | 0       | 1         | 2                | 100         |
| Specie fiorita          | 25              | 0       | 1         | 2                | 50          |
| Storico prezzi          | 50              | 0       | 1         | 2                | 100         |
| <b>Totale</b>           |                 |         |           |                  | <b>500</b>  |

L'inserimento di una nuova specie coinvolge più strutture dati. La tabella CodiceSpecieProgressivo viene utilizzata per ottenere e aggiornare atomicamente il progressivo necessario alla generazione del codice univoco. Successivamente viene inserita la nuova specie in SpecieDiPianta e, nel caso di una specie fiorita, anche in SpecieFiorita.

Infine, viene sempre inserita una nuova registrazione in StoricoPrezzi contenente il prezzo iniziale della specie. Tutte queste operazioni vengono eseguite all'interno della stessa transazione mediante la procedura InserisciSpecieConPrezzo.

### I2 Inserisci nuova azienda rivenditrice

| Entità               | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|----------------------|-----------------|---------|-----------|------------------|-------------|
| Azienda rivenditrice | 30              | 1       | 1         | 3                | 90          |

L'inserimento di una nuova azienda richiede sia una lettura, necessaria alla verifica della partita IVA per evitare duplicati, sia una scrittura per registrare i dati anagrafici.

### I3 Inserisci nuovo fornitore

| Entità / relazione | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|--------------------|-----------------|---------|-----------|------------------|-------------|
| Fornitore          | 5               | 1       | 1         | 3                | 15          |

Analogamente al caso delle aziende rivenditrici, anche l'inserimento di un nuovo fornitore comporta una lettura per il controllo del codice fiscale e una scrittura per l'inserimento.

**I4 Inserisci nuovo ordine di vendita**

| Entità               | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|----------------------|-----------------|---------|-----------|------------------|-------------|
| Ordine di vendita    | 4000            | 0       | 1         | 2                | 8000        |
| Azienda rivenditrice | 4000            | 1       | 0         | 1                | 4000        |
| Costo totale         |                 |         |           |                  | 12000       |

L'inserimento di un ordine di vendita prevede una scrittura nella tabella degli ordini e una lettura sull'anagrafica delle aziende rivenditrici, necessaria alla verifica della partita IVA.

**I5 Inserisci dettaglio ordine di vendita**

| Entità                      | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|-----------------------------|-----------------|---------|-----------|------------------|-------------|
| Dettaglio ordine di vendita | 28000           | 0       | 1         | 2                | 56000       |
| Giacenza in magazzino       | 28000           | 1       | 0         | 1                | 28000       |
| Totale                      |                 |         |           |                  | 84000       |

Per ogni riga dell'ordine viene verificata la disponibilità della specie in magazzino. Il prezzo unitario, ricevuto insieme ai dati della riga d'ordine, viene validato e successivamente memorizzato come PrezzoUnitario nel dettaglio, costituendo una copia del prezzo valida per l'intero ciclo di vita dell'ordine. Eventuali variazioni successive dello storico dei prezzi non modificano le righe già registrate.

**I6 Inserisci nuovo ordine di rifornimento**

| Entità / relazione        | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|---------------------------|-----------------|---------|-----------|------------------|-------------|
| Ordine di rifornimento    | 400             | 0       | 1         | 2                | 800         |
| Fornitore fornisce specie | 400             | 1       | 0         | 1                | 400         |
| Costo totale              |                 |         |           |                  | 1200        |

L'inserimento di un nuovo ordine di rifornimento prevede una scrittura nella tabella degli ordini di rifornimento, accompagnata da una lettura per individuare il fornitore che fornisce la specie desiderata.

**I7 Inserisci dettaglio ordine di rifornimento**

| Entità                           | Frequenza annua | Lecture | Scritture | Costo operazione | Costo annuo |
|----------------------------------|-----------------|---------|-----------|------------------|-------------|
| Dettaglio ordine di rifornimento | 4000            | 0       | 1         | 2                | 8000        |
| Fornitore fornisce specie        | 4000            | 1       | 0         | 1                | 4000        |
| Totale                           |                 |         |           |                  | 12000       |

Si stima un volume annuo di circa 400 ordini di rifornimento, ciascuno dei quali comprende mediamente dieci specie di piante, corrispondenti quindi a dieci righe di dettaglio per ordine.

Per ogni riga inserita viene verificata la validità dei dati e controllato che la specie richiesta sia effettivamente fornita dal fornitore selezionato.

Solo a seguito di queste verifiche l'inserimento viene confermato e registrato nel sistema.

**I8 Inserisci variazione di prezzo**

| Entità                                    | Frequenza annua | Lecture | Scritture | Costo operazione | Costo annuo |
|-------------------------------------------|-----------------|---------|-----------|------------------|-------------|
| Specie di pianta                          | 1500            | 1       | 0         | 1                | 1500        |
| Storico prezzi – recupero prezzo corrente | 1500            | 1       | 0         | 1                | 1500        |
| Storico prezzi – nuova variazione         | 1500            | 0       | 1         | 2                | 3000        |
| <b>Totale</b>                             |                 |         |           |                  | <b>6000</b> |

Ogni variazione di prezzo richiede una lettura sulla specie per verificarne l'esistenza, una lettura sullo storico dei prezzi per recuperare il prezzo attualmente in vigore e una nuova scrittura nello storico. La variazione non sovrascrive la registrazione precedente, ma genera sempre una nuova riga, preservando l'intera evoluzione temporale del prezzo.

**U1    Aggiorna giacenza magazzino**

| Entità             | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|--------------------|-----------------|---------|-----------|------------------|-------------|
| Giacenza magazzino | 32000           | 1       | 1         | 3                | 96000       |

L'aggiornamento della giacenza richiede sempre la lettura del valore attuale, seguita dalla scrittura del nuovo valore aggiornato. Per questo motivo il costo complessivo è stato stimato pari a 3.

**U2    Aggiorna stato ordine di vendita**

| Entità            | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|-------------------|-----------------|---------|-----------|------------------|-------------|
| Ordine di vendita | 12000           | 1       | 1         | 3                | 36000       |

Considerando circa 4.000 ordini di vendita all'anno e ipotizzando in media tre variazioni di stato per ciascun ordine, si ottengono circa 12.000 aggiornamenti annuali. Ogni aggiornamento richiede una lettura dello stato corrente e una scrittura del nuovo stato, per un costo dell'operazione pari a 3. Il costo annuo complessivo risulta quindi pari a 36.000.

**U3    Aggiorna stato ordine di rifornimento**

| Entità                 | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|------------------------|-----------------|---------|-----------|------------------|-------------|
| Ordine di rifornimento | 1200            | 1       | 1         | 3                | 3600        |

Anche qui ogni ordine mediamente subisce tre variazioni di stato.

**R1    Visualizza catalogo specie di piante**

| Entità           | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|------------------|-----------------|---------|-----------|------------------|-------------|
| Specie di pianta | 5000            | 1       | 0         | 1                | 5000        |

**R2    Visualizza storico prezzi**

| Entità           | Frequenza annua | Letture | Scritture | Costo operazione | Costo annuo |
|------------------|-----------------|---------|-----------|------------------|-------------|
| Specie di pianta | 1500            | 1       | 0         | 1                | 1500        |

## Ristrutturazione dello schema E-R

Nel processo di ristrutturazione dello schema concettuale si è partiti da un'analisi delle ridondanze presenti nel modello iniziale.

Un primo intervento ha riguardato la gestione degli indirizzi. Nel modello originario, le aziende rivenditrici disponevano sia di un indirizzo legale che di un indirizzo di fatturazione, mentre i fornitori avevano a loro volta uno o più indirizzi. È stata introdotta un'entità autonoma denominata *INDIRIZZI*, alla quale fanno riferimento sia i fornitori sia le aziende rivenditrici.

Si è scelto di mantenere l'indirizzo di spedizione all'interno degli ordini di vendita e di rifornimento, così da preservare la tracciabilità dei dati. In questo modo eventuali modifiche agli indirizzi centrali non alterano la correttezza degli ordini passati.

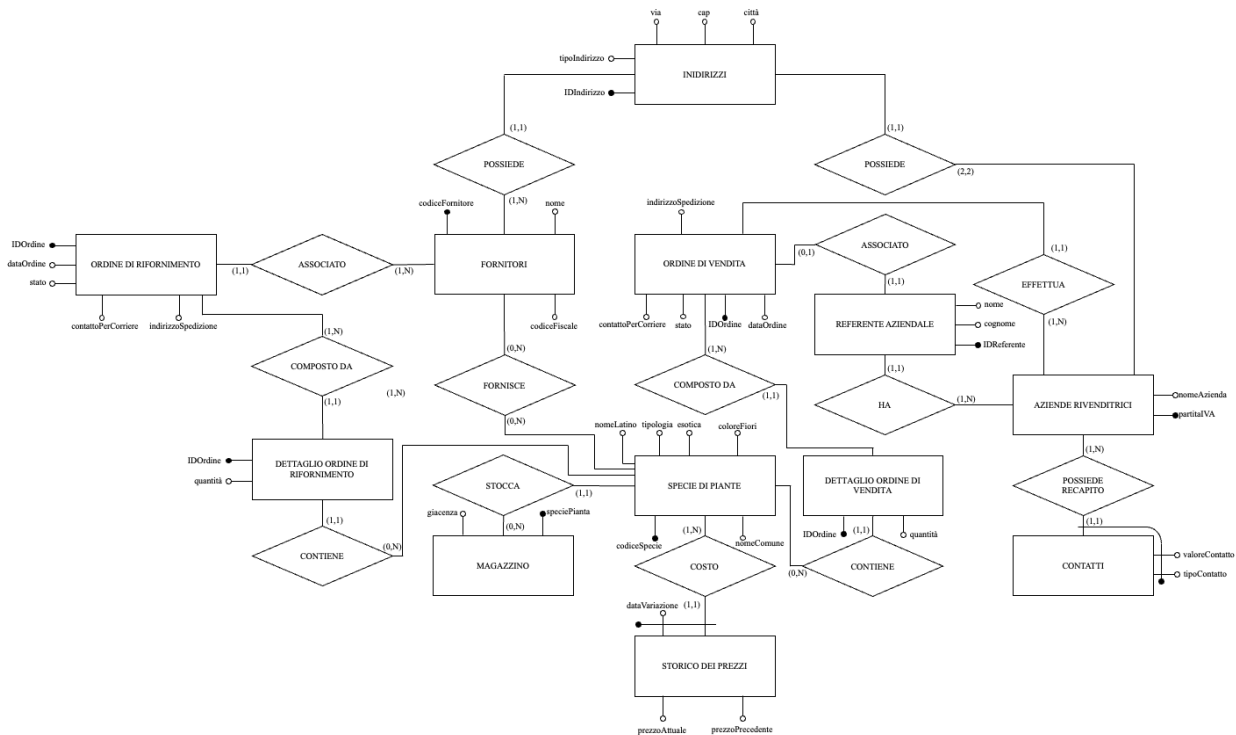
Un secondo intervento ha riguardato la rappresentazione delle specie fiorite. Nel modello relazionale si è scelto di mantenere una tabella specifica *SpecieFiorita*, collegata a *SpecieDiPianta* tramite il codice della specie. In questo modo gli attributi comuni a tutte le specie rimangono memorizzati in *SpecieDiPianta*, mentre l'informazione relativa alla colorazione viene registrata esclusivamente per le specie fiorite.

Tale soluzione evita di introdurre valori non applicabili per le specie non fiorite e mantiene separata l'informazione specifica della specializzazione.

Un'operazione simile è stata condotta per i responsabili. In precedenza, era stata introdotta un'entità generale *RESPONSABILI DI SETTORE*, con due sottoclassi: *RESPONSABILE LOGISTICO* e *RESPONSABILE COMMERCIALE*.

Un'ulteriore ristrutturazione ha riguardato i responsabili di settore. Le entità *RESPONSABILE COMMERCIALE* e *RESPONSABILE LOGISTICO*, individuate nel modello concettuale, non sono state tradotte in relazioni separate. Nell'implementazione, gli account applicativi sono rappresentati dalla relazione *UTENTI*, mentre la distinzione tra responsabile commerciale, responsabile logistico e amministratore è gestita mediante l'attributo *Ruolo*.

Infine, per quanto riguarda gli identificatori primari, per i referenti aziendali si è introdotto l'attributo *IDReferente*, che garantisce un'identificazione univoca anche in presenza di omonimie, sostituendo la chiave composta da nome e cognome.



Il risultato finale è uno schema più compatto, leggibile e coerente, che riduce le ridondanze e mantiene tutte le informazioni necessarie alla gestione del sistema di vendita all'ingrosso di Verde S.r.l.

## Trasformazione di attributi e identificatori

Gli indirizzi legale e di fatturazione delle aziende rivenditrici, inizialmente modellati come attributi dell'entità, non sono più gestiti direttamente all'interno di *AZIENDERIVENDITRICI*, ma sono stati sostituiti da riferimenti all'entità autonoma *INDIRIZZI*.

In essa sono raccolti i dati principali di un indirizzo, ed è identificata in modo univoco dall'attributo *IDIndirizzo*. Questo identificativo viene poi utilizzato come collegamento verso le aziende rivenditrici e i fornitori, diventando una chiave esterna.

Un'ulteriore trasformazione ha riguardato i referenti aziendali, per garantire l'identificazione univoca anche in presenza di omonimie, l'identificatore è stato sostituito da un codice numerico *IDReferente*, che funge da chiave primaria al posto della coppia nome-cognome.

Le relazioni *COMPOSTO-DA* sono diventate tabelle denominate *DETTAGLIO ORDINE DI VENDITA* e *DETTAGLIO ORDINE DI RIFORNIMENTO* e i riferimenti alle specie ordinate insieme alle relative quantità.

## Traduzione di entità e associazioni

A partire dallo schema E-R, le entità e le relazioni sono state tradotte in tabelle relazionali che descrivono in maniera completa il sistema informativo di verde S.r.l.

|                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------|
| SPECIEDIPIANTE ( <u>CodiceSpecie</u> , NomeComune, NomeLatino, Tipologia, Esotica, Fiorita*)                                            |
| SPECIEFIORITA ( <u>CodiceSpecie</u> , Colorazioni)                                                                                      |
| MAGAZZINO ( <u>CodiceSpecie</u> , Quantità)                                                                                             |
| FORNITORE ( <u>CodiceFornitore</u> , Nome, CodiceFiscale)                                                                               |
| FORNITURASPECIE ( <u>CodiceFornitore</u> , <u>CodiceSpecie</u> , Attiva, Quantità)                                                      |
| STORICOPREZZI ( <u>SpeciePianta</u> , <u>DataVariazione</u> , PrezzoAttuale, PrezzoPrecedente)                                          |
| AZIENDERIVENDITRICI ( <u>PartitaIVA</u> , NomeAzienda, <u>IDIndirizzoLegale</u> , <u>IDIndirizzoFatturazione</u> , <u>IDReferente</u> ) |
| INDIRIZZI ( <u>IDIndirizzo</u> , TipoIndirizzo, Via, CAP, Città)                                                                        |
| INDIRIZZIFORNITORE ( <u>CodiceFornitore</u> , <u>IDIndirizzo</u> , TipoIndirizzo)                                                       |
| CONTATTI ( <u>IDContatto</u> , TipoContatto, ValoreContatto)                                                                            |
| POSSIEDERECAPI TO ( <u>IDContatto</u> , <u>PartitaIVA</u> )                                                                             |
| ORDINIDIVENDITA (IDOrdine, PartitaIVA, DataOrdine, Stato, IndirizzoDiSpedizione, Referente, RecapitoCorriere)                           |
| REFERENTIAZIENDALI ( <u>IDReferente</u> , Nome, Cognome)                                                                                |
| DETTAGLIOORDINEDIVENDITA ( <u>IDOrdine</u> , <u>CodiceSpecie</u> , Quantità, PrezzoUnitario)                                            |
| ORDINEDIRIFORNIMENTO (IDOrdine, CodiceFornitore, DataOrdine, Stato, EmailContatto, IndirizzoDiSpedizione, Referente, RecapitoCorriere)  |
| DETTAGLIOORDINERIFORNIMENTO (IDOrdine, CodiceSpecie, Quantità)                                                                          |

## Vincoli

```

SPECIEFIORITA (CodiceSpecie) ⊆ SPECIEDIPIANTE (CodiceSpecie)

MAGAZZINO (CodiceSpecie) ⊆ SPECIEDIPIANTE (CodiceSpecie)

STORICOPREZZI (SpeciePianta) ⊆ SPECIEDIPIANTE (CodiceSpecie)

FORNITURASPECIE (CodiceSpecie) ⊆ SPECIEDIPIANTE (CodiceSpecie)

DETTAGLIOORDINEDIVENDITA (CodiceSpecie) ⊆ SPECIEDIPIANTE (CodiceSpecie)

DETTAGLIOORDINERIFORNIMENTO (CodiceSpecie) ⊆ SPECIEDIPIANTE (CodiceSpecie)

FORNITURASPECIE (CodiceFornitore) ⊆ FORNITORE (CodiceFornitore)

ORDINERIFORNIMENTO (CodiceFornitore) ⊆ FORNITORE (CodiceFornitore)

INDIRIZZOFORNITORE (CodiceFornitore) ⊆ FORNITORE (CodiceFornitore)

ORDINIDIVENDITA (PartitaIVA) ⊆ AZIENDERIVENDITRICI (PartitaIVA)

AZIENDERIVENDITRICI (IDIndirizzoLegale) ⊆ INDIRIZZI (IDIndirizzo)

AZIENDERIVENDITRICI (IDIndirizzoFatturazione) ⊆ INDIRIZZI (IDIndirizzo)

AZIENDERIVENDITRICI (IDReferente) ⊆ REFERENTIAZIENDALI (IDReferente)

INDIRIZZOFORNITORE (IDIndirizzo) ⊆ INDIRIZZI (IDIndirizzo)

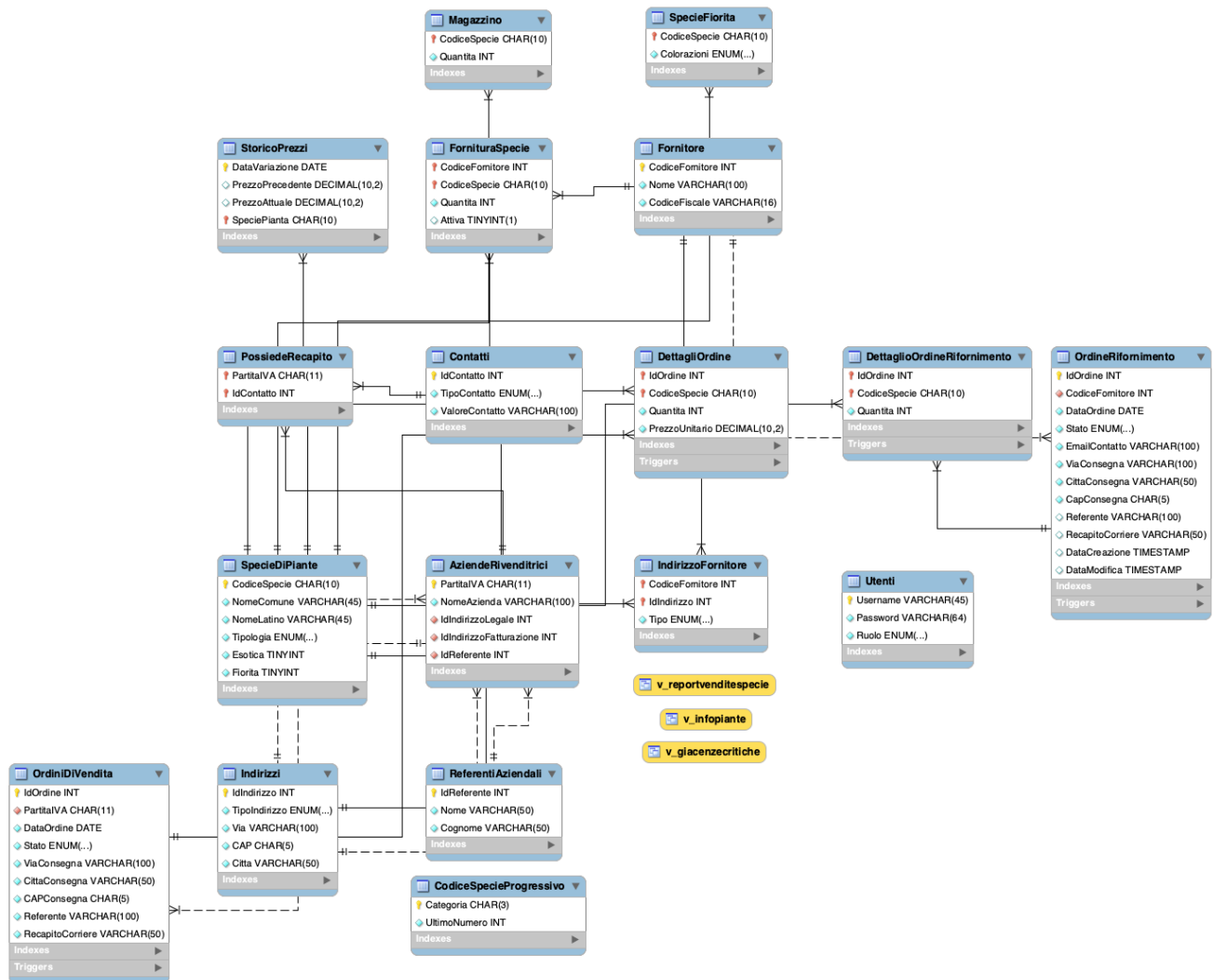
POSSIEDERECAPIATO (PartitaIVA) ⊆ AZIENDERIVENDITRICI (PartitaIVA)

POSSIEDERECAPIATO (IDContatto) ⊆ CONTATTI (IDContatto)

DETTAGLIOORDINEDIVENDITA (IDOrdine) ⊆ ORDINIDIVENDITA (IDOrdine)

DETTAGLIOORDINERIFORNIMENTO (IDOrdine) ⊆ ORDINERIFORNIMENTO (IDOrdine)

```



## Normalizzazione del modello relazionale

Il modello relazionale del database sviluppato per Verde S.r.l. rispetta la prima forma normale (1NF). Tutti gli attributi presenti nelle tabelle sono infatti atomici, non esistono valori multi-valore e ogni tabella è caratterizzata da una chiave primaria.

Il modello rispetta la seconda forma normale (2NF), perché non ci sono dipendenze parziali, ogni attributo non chiave dipende in modo completo dalla chiave primaria.

Il modello è stato progettato evitando dipendenze parziali e transitive tra gli attributi delle relazioni. Per le tabelle ORDINE DI VENDITA e ORDINE DI RIFORNIMENTO si è inoltre scelto di memorizzare direttamente alcune informazioni relative alla consegna, quali l'indirizzo e il referente. Tale scelta consente di conservare una copia dei dati validi al momento della creazione dell'ordine, evitando che successive modifiche ai dati anagrafici possano alterare le informazioni associate agli ordini già registrati.

## 5. Progettazione fisica

### Utenti e privilegi

I profili descritti di seguito sono gestiti a livello applicativo mediante la tabella UTENTI e il relativo attributo Ruolo. I comandi GRANT riportati nella sezione rappresentano invece la configurazione prevista a livello DBMS per limitare l'accesso alle sole stored procedure consentite a ciascun ruolo. Tali istruzioni di configurazione degli account e dei privilegi non fanno parte dello script SQL finale utilizzato dall'applicazione.

All'interno dell'applicazione sono stati definiti tre profili utente principali:

- responsabile commerciale;
- responsabile logistico;
- l'amministratore del sistema.

### Responsabile commerciale

Il profilo del responsabile commerciale è stato progettato per consentire la gestione delle attività legate all'area vendite e gestione delle specie di piante trattate. In particolare, a questo utente sono attribuiti i privilegi di creazione, lettura, modifica ed eliminazione (CRUD) sulle operazioni riguardanti le specie di piante, la variazione del catalogo prezzi, la gestione degli ordini di vendita e l'amministrazione delle aziende rivenditrici.

### Procedure per gestione specie di piante

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciSpecieConPrezzo TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaSpecie TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaSpecieTotali TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaSpecie TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaSpecie TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaSpeciePerNome TO
'responsabileCommerciale'@'localhost';
```

```
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaSpeciePerTipologia TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaSpeciePerColorazione TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaSpecieEsistente
TO 'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaCatalogoPiante
TO 'responsabileCommerciale'@'localhost';
```

### Procedure per gestione prezzi

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciVariazionePrezzo TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaStoricoPrezzi TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.OttieniPrezzoAttuale TO
'responsabileCommerciale'@'localhost';
```

### Procedure per gestione degli ordini

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciOrdineVenditaCompleto TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaOrdineVendita TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaOrdiniVendita TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaDettagliOrdineVendita TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaOrdineVenditaAperto TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaOrdineVendita TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaOrdineVenditaEsistente TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaStatoOrdine TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaOrdiniPerStato TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaOrdiniPerAzienda TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.OttieniQuantitaGiacenza TO
'responsabileCommerciale'@'localhost';
```

### Procedure per aziende

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciAziendaRivenditrice TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaAziendaRivenditrice TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaAziendaRivenditrice TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaAziendeClienti TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaAziendaPerPartitaIVA TO
'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaAziendaRivenditriceEsistente
TO 'responsabileCommerciale'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaContattiAzienda
TO 'responsabileCommerciale'@'localhost';
```

## Responsabile Logistico

Il responsabile logistico, invece, dispone di privilegi analoghi, ma legati all'ambito gestionale del magazzino e alle gestione dei fornitori. In questo caso, l'utente è abilitato a effettuare operazioni CRUD sulle giacenze, sugli ordini di rifornimento e sui dati relativi ai fornitori.

| RESPONSABILE LOGISTICO             |  |
|------------------------------------|--|
| Menù Responsabile Logistico        |  |
| 1. Gestisci Magazzino              |  |
| 2. Gestisci Ordini di Rifornimento |  |
| 3. Gestisci Fornitori              |  |
| 4. Esci                            |  |

## Procedure per giacenze

```
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaSpecieEsistente
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaCatalogoPiante
TO 'responsabileLogistico'@'localhost';GRANT EXECUTE ON PROCEDURE
verde_srl.InserisciGiacenza TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaGiacenza TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.OttieniQuantitaGiacenza TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaGiacenzeCritiche TO
'responsabileLogistico'@'localhost';
```

## Procedure per fornitori

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciFornitoreCompleto
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaFornitoreCompleto
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaFornitore
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaFornitorePerCodice
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaFornitori
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaFornitoreEsistente
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaIndirizziFornitore
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciFornituraSpecie
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaFornituraSpecie
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.SpecieForniteDaFornitore
TO 'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaFornituraSpecie
TO 'responsabileLogistico'@'localhost';
```

## Procedure per ordini di rifornimento

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciOrdineRifornimentoCompleto TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaOrdineRifornimento TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaOrdiniRifornimento TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaDettagliOrdineRifornimento TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaOrdineRifornimentoAperto TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaOrdineRifornimento TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaOrdineRifornimentoEsistente TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.ModificaStatoOrdineRifornimento TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaOrdiniRifornimentoPerStato TO
'responsabileLogistico'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.RicercaOrdiniRifornimentoPerFornitore TO
'responsabileLogistico'@'localhost';
```

## Amministratore del sistema

Infine, l'amministratore del sistema rappresenta un profilo dedicato alla gestione degli account applicativi. A livello di database gli vengono concessi esclusivamente i privilegi necessari per la creazione, visualizzazione ed eliminazione degli utenti e per le operazioni di verifica richieste dal client Java. L'amministratore non dispone pertanto di privilegi generali sulle funzionalità commerciali o logistiche.

Tale configurazione applica il principio del minimo privilegio, limitando ciascun account del DBMS alle sole stored procedure necessarie allo svolgimento delle operazioni previste dal proprio ruolo.



### Menù Amministratore

1. Gestisci Utenti
2. Esci

## Procedure per gestione utenti

```
GRANT EXECUTE ON PROCEDURE verde_srl.InserisciUtente TO 'amministratore'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VisualizzaUtenti TO 'amministratore'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.EliminaUtente TO 'amministratore'@'localhost';
GRANT EXECUTE ON PROCEDURE verde_srl.VerificaUsernameEsistente TO
'amministratore'@'localhost';
```

## Strutture di memorizzazione

| Tabella AziendeRivenditrici |               |                        |
|-----------------------------|---------------|------------------------|
| Colonna                     | Tipo di dato  | Attributi <sup>2</sup> |
| PartitaIVA                  | CHAR (11)     | PK, NN                 |
| NomeAzienda                 | VARCHAR (100) | NN                     |
| IdIndirizzoLegale           | INT           | NN                     |
| IdIndirizzoFatturazione     | INT           | NN                     |
| IdReferente                 | INT           | NN                     |

| Tabella CodiceSpecieProgressivo |              |           |
|---------------------------------|--------------|-----------|
| Colonna                         | Tipo di dato | Attributi |
| Categoria                       | CHAR (3)     | PK, NN    |
| UltimoNumero                    | INT          | NN        |

| Tabella Contatti |                                        |           |
|------------------|----------------------------------------|-----------|
| Colonna          | Tipo di dato                           | Attributi |
| IdContatto       | INT                                    | PK, NN    |
| TipoContatto     | ENUM('TELEFONO', 'CELLULARE', 'EMAIL') | NN        |
| ValoreContatto   | VARCHAR (100)                          | NN        |

| Tabella DettagliOrdineRifornimento |              |           |
|------------------------------------|--------------|-----------|
| Colonna                            | Tipo di dato | Attributi |
| IdOrdine                           | INT          | PK, NN    |
| CodiceSpecie                       | CHAR (10)    | PK, NN    |
| Quantità                           | INT          | NN        |

| Tabella DettagliOrdine |                |           |
|------------------------|----------------|-----------|
| Colonna                | Tipo di dato   | Attributi |
| IdOrdine               | INT            | PK, NN    |
| CodiceSpecie           | CHAR (10)      | PK, NN    |
| Quantità               | INT            | NN        |
| PrezzoUnitario         | DECIMAL (10,2) | NN        |

| Tabella Fornitore |               |           |
|-------------------|---------------|-----------|
| Colonna           | Tipo di dato  | Attributi |
| CodiceFornitore   | INT           | PK, NN    |
| Nome              | VARCHAR (100) | NN        |
| CodiceFiscale     | CHAR (16)     | NN, UQ    |

<sup>2</sup> PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

| Tabella FornituraSpecie |              |                 |
|-------------------------|--------------|-----------------|
| Colonna                 | Tipo di dato | Attributi       |
| CodiceFornitore         | INT          | PK, NN          |
| CodiceSpecie            | CHAR (10)    | PK, NN          |
| Quantità                | INT          | NN, DEFAULT '0' |
| Attiva                  | TINYINT (1)  | DEFAULT '1'     |

| Tabella Indirizzi |                                |           |
|-------------------|--------------------------------|-----------|
| Colonna           | Tipo di dato                   | Attributi |
| IdIndirizzo       | INT                            | PK, NN    |
| TipoIndirizzo     | ENUM('LEGALE', 'FATTURAZIONE') | NN        |
| Via               | VARCHAR (100)                  | NN        |
| CAP               | CHAR (5)                       | NN        |
| Città             | VARCHAR (50)                   | NN        |

| Tabella IndirizzoFornitore |                                |           |
|----------------------------|--------------------------------|-----------|
| Colonna                    | Tipo di dato                   | Attributi |
| CodiceFornitore            | INT                            | PK, NN    |
| IdIndirizzo                | INT                            | PK, NN    |
| Tipo                       | ENUM('LEGALE', 'FATTURAZIONE') | NN        |

| Tabella Magazzino |              |           |
|-------------------|--------------|-----------|
| Colonna           | Tipo di dato | Attributi |
| CodiceSpecie      | CHAR (10)    | PK, NN    |
| Quantità          | INT          | NN        |

| Tabella OrdineRifornimento |                                                                    |                                     |
|----------------------------|--------------------------------------------------------------------|-------------------------------------|
| Colonna                    | Tipo di dato                                                       | Attributi                           |
| IdOrdine                   | INT                                                                | PK, NN, AI                          |
| CodiceFornitore            | INT                                                                | NN                                  |
| DataOrdine                 | DATE                                                               | NN                                  |
| Stato                      | ENUM('APERTO', 'CONFERMATO', 'SPEDITO', 'CONSEGNATO', 'ANNULLATO') | NN, DEFAULT 'APERTO'                |
| EmailContatto              | VARCHAR (100)                                                      | NN, DEFAULT 'magazzino@verdesrl.it' |
| ViaConsegna                | VARCHAR (100)                                                      | NN, DEFAULT 'Via delle piante 10'   |
| CittaConsegna              | VARCHAR (50)                                                       | NN, DEFAULT 'Roma'                  |
| CapConsegna                | CHAR (5)                                                           | NN, DEFAULT '00100'                 |
| Referente                  | VARCHAR (100)                                                      |                                     |
| RecapitoCorriere           | VARCHAR (50)                                                       |                                     |
| DataCreazione              | TIMESTAMP                                                          | CURRENT_TIMESTAMP                   |

|              |           |                                                     |
|--------------|-----------|-----------------------------------------------------|
| DataModifica | TIMESTAMP | CURRENT_TIMESTAMP ON<br>UPDATE<br>CURRENT_TIMESTAMP |
|--------------|-----------|-----------------------------------------------------|

| Tabella OrdineDiVendita |                                                                             |                      |
|-------------------------|-----------------------------------------------------------------------------|----------------------|
| Colonna                 | Tipo di dato                                                                | Attributi            |
| IdOrdine                | INT                                                                         | PK, NN, AI           |
| PartitaIVA              | CHAR (11)                                                                   | NN                   |
| DataOrdine              | DATE                                                                        | NN                   |
| Stato                   | ENUM('APERTO',<br>'CONFERMATO', 'SPEDITO',<br>'CONSEGNATO',<br>'ANNULLATO') | NN, DEFAULT 'APERTO' |
| ViaConsegna             | VARCHAR (100)                                                               | NN                   |
| CittaConsegna           | VARCHAR (50)                                                                | NN                   |
| CapConsegna             | CHAR (5)                                                                    | NN                   |
| Referente               | VARCHAR (100)                                                               | NN                   |
| RecapitoCorriere        | VARCHAR (50)                                                                | NN                   |

| Tabella PossiedeRecapito |              |           |
|--------------------------|--------------|-----------|
| Colonna                  | Tipo di dato | Attributi |
| PartitaIVA               | CHAR (11)    | PK, NN    |
| IdContatto               | INT          | PK, NN    |

| Tabella ReferentiAziendali |              |           |
|----------------------------|--------------|-----------|
| Colonna                    | Tipo di dato | Attributi |
| IdReferente                | INT          | PK, NN    |
| Nome                       | VARCHAR (50) | NN        |
| Cognome                    | VARCHAR (50) | NN        |

| Tabella SpecieDiPiante |                                     |                 |
|------------------------|-------------------------------------|-----------------|
| Colonna                | Tipo di dato                        | Attributi       |
| CodiceSpecie           | CHAR (10)                           | PK, NN          |
| NomeComune             | VARCHAR (45)                        | NN              |
| NomeLatino             | VARCHAR (45)                        | NN              |
| Tipologia              | ENUM('APPARTAMENTO',<br>'GIARDINO') | NN              |
| Esotica                | TINYINT                             | NN, DEFAULT '0' |
| Fiorita                | TINYINT                             | NN              |

| Tabella SpecieFiorita |                                                                               |           |
|-----------------------|-------------------------------------------------------------------------------|-----------|
| Colonna               | Tipo di dato                                                                  | Attributi |
| CodiceSpecie          | CHAR (10)                                                                     | PK, NN    |
| Colorazioni           | ENUM('ROSSA', 'GIALLA', 'ARANCIONE', 'BLU', 'VIOLA', 'BIANCA', 'MULTICOLORE') | NN        |

| Tabella StoricoPrezzi |                |                |
|-----------------------|----------------|----------------|
| Colonna               | Tipo di dato   | Attributi      |
| DataVariazione        | DATETIME(6)    | PK, NN         |
| SpeciePianta          | CHAR (10)      | PK, NN         |
| PrezzoAttuale         | DECIMAL (10,2) | NN             |
| PrezzoPrecedente      | DECIMAL (10,2) | DEFAULT 'NULL' |

L'attributo DataVariazione è stato definito come DATETIME(6) anziché DATE, in modo da rappresentare l'istante preciso in cui viene effettuata una variazione di prezzo e consentire più variazioni sulla stessa specie nella medesima giornata. La chiave primaria rimane composta da DataVariazione e SpeciePianta. PrezzoAttuale è obbligatorio, mentre PrezzoPrecedente può assumere valore NULL esclusivamente in corrispondenza del primo prezzo registrato per una specie.

| Tabella Utenti |                                                                            |           |
|----------------|----------------------------------------------------------------------------|-----------|
| Colonna        | Tipo di dato                                                               | Attributi |
| Username       | VARCHAR (45)                                                               | PK, NN    |
| Password       | VARCHAR (64)                                                               | NN        |
| Ruolo          | ENUM('amministratore', 'responsabileCommerciale', 'responsabileLogistico') | NN        |

## Indici

| Tabella SpecieFiorita                |       |
|--------------------------------------|-------|
| Indice idx_SpecieFiorita_Colorazioni | Tipo: |
| Colorazioni                          | IDX   |

L'indice è stato creato per ottimizzare le operazioni di ricerca e di filtro basate sulla colorazione delle specie fiorite. In questo modo, quando l'utente deve individuare tutte le specie con un determinato colore, non è necessario effettuare una scansione completa della tabella, l'indice consente di ridurre di molto i tempi di accesso ai dati.

| Tabella SpecieDiPiante                      |       |
|---------------------------------------------|-------|
| Indice idx_SpecieDiPiante_Tipologia_Fiorita | Tipo: |
| Colonna 1: Tipologia                        | IDX   |
| Colonna 2: Fiorita                          | IDX   |

L'indice composto è stato introdotto per velocizzare le query di ricerca sul catalogo delle specie, in particolare quelle che filtrano le piante in base alla tipologia (appartamento o giardino) e alla caratteristica di essere o meno fiorite.

| Tabella SpecieDiPiante            |       |
|-----------------------------------|-------|
| Indice idx_SpecieDiPiante_Esotica | Tipo: |
| Esotica                           | IDX   |

L'indice sulla colonna Esotica è stato creato per agevolare le ricerche relative alle sole specie esotiche.

| Tabella OrdiniDiVendita          |       |
|----------------------------------|-------|
| Indice idx_OrdiniDiVendita_Stato | Tipo: |
| Stato                            | IDX   |

L'indice consente di velocizzare le interrogazioni sugli ordini in base al loro stato attuale. Poiché la gestione e il cambio di stato dell'ordine rappresentano operazioni ricorrenti nel sistema, l'introduzione di questo indice permette di migliorare di molto le prestazioni.

| Tabella OrdiniDiVendita               |       |
|---------------------------------------|-------|
| Indice idx_OrdiniDiVendita_PartitaIVA | Tipo: |
| PartitaIVA                            | IDX   |

Questo indice è stato inserito per ottimizzare le interrogazioni che recuperano gli ordini associati a una specifica azienda cliente. L'uso dell'indice consente di visualizzare rapidamente tutti gli ordini legati a una determinata partita IVA.

| Tabella StoricoPrezzi                |       |
|--------------------------------------|-------|
| Indice idx_StoricoPrezzi_Specie_Data | Tipo: |
| Colonna 1: SpeciePianta              | IDX   |
| Colonna 2: DataVariazione            | IDX   |

L'indice composto su SpeciePianta e DataVariazione è stato progettato per migliorare le interrogazioni che restituiscono l'andamento dei prezzi nel tempo per una determinata specie di pianta.

## Trigger

L'utilizzo dei trigger si è reso necessario per garantire il rispetto delle regole aziendali. Essi intervengono in maniera automatica durante le operazioni di inserimento, aggiornamento e cancellazione, garantendo che i dati risultino coerenti.

### Trigger riguardanti gli ordini di vendita

I trigger **after\_dettaglio\_ordine\_insert** e **after\_dettaglio\_ordine\_delete** garantiscono l'aggiornamento immediato delle giacenze in magazzino ogni volta che una riga viene aggiunta o rimossa da un ordine, mantenendo in questo modo l'allineamento con la regola di derivazione RDR1

```
CREATE TRIGGER `after_dettaglio_ordine_insert`
AFTER INSERT ON `DettagliOrdine`
FOR EACH ROW BEGIN
    DECLARE v_stato ENUM('APERTO', 'CONFERMATO', 'SPEDITO', 'CONSEGNATO', 'ANNULLATO');
    SELECT Stato INTO v_stato
    FROM OrdiniDiVendita
    WHERE IdOrdine = NEW.IdOrdine;

    IF v_stato = 'APERTO' THEN
        UPDATE `Magazzino`
        SET Quantita = Quantita - NEW.Quantita
        WHERE CodiceSpecie = NEW.CodiceSpecie;
    END IF; END
//
```

```
CREATE TRIGGER `after_dettaglio_ordine_delete`
AFTER DELETE ON `DettagliOrdine`
FOR EACH ROW BEGIN
    DECLARE v_stato ENUM('APERTO', 'CONFERMATO', 'SPEDITO', 'CONSEGNATO', 'ANNULLATO');
    SELECT Stato INTO v_stato
    FROM OrdiniDiVendita
    WHERE IdOrdine = OLD.IdOrdine;
    IF v_stato = 'CONFERMATO' OR
v_stato = 'APERTO' THEN
        UPDATE `Magazzino`
        SET Quantita = Quantita +
OLD.Quantita
        WHERE CodiceSpecie =
OLD.CodiceSpecie;
    END IF;
END //
```

Il trigger **before\_ordine\_stato\_update** è stato progettato per garantire coerenza nella gestione del ciclo di vita degli ordini di vendita. Ogni volta che lo stato di un ordine viene modificato, il trigger controlla che la transizione sia consentita, ovvero non è possibile alterare ordini già consegnati o annullati, un ordine può essere confermato solo se si trova in stato aperto, può essere spedito soltanto se già confermato e consegnato solo se già spedito. La disponibilità di magazzino non viene verificata nuovamente durante la conferma, poiché è già controllata all'apertura dell'ordine e le quantità richieste risultano già impegnate mediante l'aggiornamento della giacenza.

```
CREATE TRIGGER before_ordine_stato_update
BEFORE UPDATE ON OrdiniDiVendita
FOR EACH ROW
BEGIN
    IF OLD.Stato <> NEW.Stato THEN

        IF OLD.Stato IN ('CONSEGNATO', 'ANNULLATO') THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Non è possibile modificare un ordine già completato o annullato';
        END IF;

        IF NEW.Stato = 'CONFERMATO'
            AND OLD.Stato <> 'APERTO' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Puoi confermare solo un ordine APERTO';
        END IF;

        IF NEW.Stato = 'SPEDITO'
            AND OLD.Stato <> 'CONFERMATO' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Puoi spedire solo un ordine confermato';
        END IF;

        IF NEW.Stato = 'CONSEGNATO'
            AND OLD.Stato <> 'SPEDITO' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Puoi consegnare solo un ordine spedito';
        END IF;

    END IF;
END$$

DELIMITER ;
```

Il trigger **after\_ordine\_stato\_update** gestisce il ripristino delle giacenze quando un ordine di vendita viene annullato. Se l'ordine passa allo stato ANNULLATO mentre si trova in stato APERTO o CONFERMATO, le quantità presenti nelle righe di dettaglio vengono nuovamente aggiunte al magazzino, poiché non devono più risultare impegnate dall'ordine.

```
CREATE TRIGGER `after_ordine_stato_update`  
AFTER UPDATE ON `OrdiniDiVendita`  
FOR EACH ROW  
BEGIN  
    IF NEW.Stato = 'ANNULLATO'  
        AND OLD.Stato IN ('APERTO', 'CONFERMATO') THEN  
  
        UPDATE Magazzino g  
        JOIN DettagliOrdine d  
        ON g.CodiceSpecie = d.CodiceSpecie  
        SET g.Quantita = g.Quantita + d.Quantita  
        WHERE d.IdOrdine = NEW.IdOrdine;  
  
    END IF;  
END;
```

## Trigger riguardanti gli ordini di rifornimento

Un secondo insieme di trigger agisce invece sulle tabelle OrdineRifornimento, DettaglioOrdineRifornimento, FornituraSpecie e Magazzino. In questo caso i controlli servono a verificare che le quantità ordinate siano positive, che il fornitore selezionato possa effettivamente fornire la specie richiesta e che le giacenze vengano aggiornate solo al momento della consegna dell'ordine.

Il trigger **before\_dettaglio\_rifornimento\_insert**, attivato prima dell'inserimento di una nuova riga nella tabella DettaglioOrdineRifornimento, controlla che la quantità richiesta sia maggiore di zero e che il fornitore scelto sia effettivamente abilitato a fornire la specie di pianta indicata.

```
CREATE TRIGGER `before_dettaglio_rifornimento_insert`
BEFORE INSERT ON `DettaglioOrdineRifornimento`
FOR EACH ROW BEGIN
  DECLARE v_stato ENUM('APERTO','CONFERMATO','SPEDITO','CONSEGNATO','ANNULLATO');
  DECLARE v_fornitore_disponibile INT DEFAULT 0;
  DECLARE v_codice_fornitore INT;
  DECLARE msg VARCHAR(255);

  SELECT Stato, CodiceFornitore INTO v_stato, v_codice_fornitore
  FROM OrdineRifornimento
  WHERE IdOrdine = NEW.IdOrdine;

  IF v_stato IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Ordine di rifornimento non trovato';
  END IF;

  IF NEW.Quantita <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'La quantità deve essere maggiore di zero';
  END IF;

  SELECT COUNT(*) INTO v_fornitore_disponibile
  FROM FornituraSpecie
  WHERE CodiceFornitore = v_codice_fornitore
    AND CodiceSpecie = NEW.CodiceSpecie
    AND Attiva = TRUE;

  IF v_fornitore_disponibile = 0 THEN
    SET msg = CONCAT('Il fornitore non pu  fornire la specie: ', NEW.CodiceSpecie);
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = msg;
  END IF;
END //
```

Il corretto avanzamento degli ordini attraverso le diverse fasi è garantito dal trigger **before\_ordine\_rifornimento\_stato\_update**, stessa modalità degli ordini di vendita.

Infine, il trigger **after\_ordine\_rifornimento\_stato\_update** gestisce l'aggiornamento delle giacenze al momento del passaggio allo stato CONSEGNA TO: tutte le quantità previste nei dettagli dell'ordine vengono aggiunte al magazzino, mantenendo così coerenti le giacenze.

```
CREATE TRIGGER `before_ordine_rifornimento_stato_update`
BEFORE UPDATE ON `OrdineRifornimento`
FOR EACH ROW
BEGIN
    IF OLD.Stato <> NEW.Stato THEN
        IF (OLD.Stato = 'CONSEGNA TO' OR OLD.Stato = 'ANNULLATO') THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Non è possibile modificare un ordine già completato o annullato';
        END IF;
        IF NEW.Stato = 'CONFERMATO' AND OLD.Stato NOT IN ('APERTO') THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Puoi confermare solo un ordine APERTO';
        END IF;
        IF NEW.Stato = 'SPEDITO' AND OLD.Stato NOT IN ('CONFERMATO') THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Puoi spedire solo un ordine CONFERMATO';
        END IF;
        IF NEW.Stato = 'CONSEGNA TO' AND OLD.Stato NOT IN ('SPEDITO') THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Puoi consegnare solo un ordine SPEDITO';
        END IF;

        IF NEW.Stato = 'CONFERMATO' THEN
            IF NOT EXISTS (SELECT 1 FROM DettaglioOrdineRifornimento WHERE IdOrdine = NEW.IdOrdine) THEN
                SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Non è possibile confermare un ordine senza dettagli';
            END IF;
        END IF;
    END IF;
END //
```

```
CREATE TRIGGER `after_ordine_rifornimento_stato_update`
AFTER UPDATE ON `OrdineRifornimento`
FOR EACH ROW
BEGIN
    IF OLD.Stato != 'CONSEGNA TO' AND NEW.Stato = 'CONSEGNA TO' THEN
        INSERT INTO `Magazzino` (CodiceSpecie, Quantita)
        SELECT CodiceSpecie, Quantita
        FROM DettaglioOrdineRifornimento
        WHERE IdOrdine = NEW.IdOrdine
        ON DUPLICATE KEY UPDATE
        Quantita = `Magazzino`.Quantita + VALUES(Quantita);
    END IF;
    IF OLD.Stato = 'CONSEGNA TO' AND NEW.Stato != 'CONSEGNA TO' THEN
        UPDATE `Magazzino` g
        JOIN DettaglioOrdineRifornimento d ON g.CodiceSpecie = d.CodiceSpecie
        SET g.Quantita = GREATEST(0, g.Quantita - d.Quantita)
        WHERE d.IdOrdine = NEW.IdOrdine;
    END IF;
END //
```

## Viste

La prima vista creata denominata **v\_InfoPiante** ha lo scopo di raccogliere in un'unica interrogazione tutte le informazioni riguardanti le specie gestite, per mostrare il catalogo completo con prezzi e giacenze. Essa unisce i dati delle specie di piante, di quelle fiorite, il prezzo attuale e le giacenze presenti in magazzino.

```
CREATE OR REPLACE VIEW `v_InfoPiante` AS
SELECT
  `s`.`CodiceSpecie`,
  `s`.`NomeComune`,
  `s`.`NomeLatino`,
  `s`.`Tipologia`,
  `s`.`Esotica`,
  `s`.`Fiorita`,
  `sf`.`Colorazioni`,
  `p`.`PrezzoAttuale`,
  IFNULL(`g`.`Quantita`, 0) AS `Giacenza`
FROM `SpecieDiPiante` `s`
LEFT JOIN `SpecieFiorita` `sf` ON `s`.`CodiceSpecie` = `sf`.`CodiceSpecie`
LEFT JOIN (
  SELECT `sp`.`SpeciePianta`, `sp`.`PrezzoAttuale`
  FROM `StoricoPrezzi` `sp`
  JOIN (
    SELECT `SpeciePianta`, MAX(`DataVariazione`) AS `MaxData`
    FROM `StoricoPrezzi`
    GROUP BY `SpeciePianta`
  ) `last_sp`
  ON `sp`.`SpeciePianta` = `last_sp`.`SpeciePianta`
  AND `sp`.`DataVariazione` = `last_sp`.`MaxData`
) `p` ON `s`.`CodiceSpecie` = `p`.`SpeciePianta`
LEFT JOIN `Magazzino` `g` ON `s`.`CodiceSpecie` = `g`.`CodiceSpecie`;
```

La seconda vista è denominata **v\_GiacenzeCritiche** e ha lo scopo di evidenziare le specie di piante con una giacenza sotto una soglia minima prefissata, che è fissata a dieci unità. Vista utile per il responsabile logistico per identificare le specie sotto giacenza minima che necessitano di un rifornimento.

```
CREATE OR REPLACE VIEW `v_GiacenzeCritiche` AS
SELECT
  `p`.`CodiceSpecie`,
  `p`.`NomeComune`,
  `p`.`NomeLatino`,
  `g`.`Quantita`
FROM `Magazzino` `g`
JOIN `SpecieDiPiante` `p` ON `g`.`CodiceSpecie` = `p`.`CodiceSpecie` WHERE `g`.`Quantita`
< 10;
```

Infine, la terza vista **v\_ReportVenditeSpecie** è stata progettata per fare un report per le specie di piante vendute. Essa riporta per ciascuna specie di pianta la quantità complessivamente venduta e il valore economico totale generato dalle vendite, includendo solamente gli ordini già spediti e consegnati, in modo da fornire un quadro dell'andamento commerciale. Utile per vedere quali sono le specie più richieste.

```
CREATE OR REPLACE VIEW `v_ReportVenditeSpecie` AS
SELECT
  `s`.`CodiceSpecie`,
  `s`.`NomeComune`,
  `s`.`Tipologia`,
  `s`.`Fiorita`,
  SUM(`d`.`Quantita`) AS `QuantitaVenduta`,
  SUM(`d`.`Quantita` * `d`.`PrezzoUnitario`) AS `ValoreTotaleVendite` FROM
  `SpecieDiPianta` `s`
  JOIN `DettagliOrdine` `d` ON `s`.`CodiceSpecie` = `d`.`CodiceSpecie`
  JOIN `OrdiniDiVendita` `o` ON `d`.`IdOrdine` = `o`.`IdOrdine`
WHERE `o`.`Stato` IN ('CONSEGNATO', 'SPEDITO')
GROUP BY `s`.`CodiceSpecie`, `s`.`NomeComune`, `s`.`Tipologia`, `s`.`Fiorita`;
```

## Funzioni

La funzione **CalcolaValoreTotaleOrdine** calcola il valore complessivo di un ordine di vendita sommando, per tutte le relative righe di dettaglio, il prodotto tra quantità e prezzo unitario. In assenza di righe restituisce 0.

La funzione **VerificaGiacenzeSufficienti** verifica se la quantità disponibile in magazzino per una determinata specie è almeno pari alla quantità richiesta. Se la specie non dispone di una giacenza registrata, la funzione restituisce FALSE.

## Stored Procedures e transazioni

### Procedura InserisciSpecieConPrezzo

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `InserisciSpecieConPrezzo`(
  IN p_nome_comune VARCHAR(45),
  IN p_nome_latino VARCHAR(45),
  IN p_tipologia ENUM('APPARTAMENTO', 'GIARDINO'),
  IN p_esotica TINYINT,
  IN p_fiorita TINYINT,
  IN p_colorazione ENUM('ROSSA', 'GIALLA', 'ARANCIONE', 'BLU', 'VIOLA', 'BIANCA',
'MULTICOLORE'),
  IN p_prezzo_iniziale DECIMAL(10,2),
  OUT p_codice_specie CHAR(10)
)
BEGIN
  DECLARE v_tipo_fioritura CHAR(1);
  DECLARE v_tipo_ambiente CHAR(2);
  DECLARE v_categoria CHAR(3);
  DECLARE v_nuovo_numero INT;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SET p_codice_specie = NULL;
    RESIGNAL;
  END;

  IF p_prezzo_iniziale IS NULL OR p_prezzo_iniziale <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Il prezzo iniziale deve essere maggiore di zero.';
  END IF;

  IF p_fiorita = 1 AND p_colorazione IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Colorazione obbligatoria per specie fiorita.';
  END IF;

  SET v_tipo_fioritura = IF(p_fiorita = 1, 'F', 'N');
  SET v_tipo_ambiente = IF(p_tipologia = 'APPARTAMENTO', 'IN', 'ES');
  SET v_categoria = CONCAT(v_tipo_fioritura, v_tipo_ambiente);

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  INSERT INTO CodiceSpecieProgressivo (Categoria, UltimoNumero)
  VALUES (v_categoria, 0)
  ON DUPLICATE KEY UPDATE Categoria = VALUES(Categoria);

  SELECT UltimoNumero + 1
  INTO v_nuovo_numero
  FROM CodiceSpecieProgressivo
  WHERE Categoria = v_categoria
  FOR UPDATE;

  UPDATE CodiceSpecieProgressivo
  SET UltimoNumero = v_nuovo_numero
  WHERE Categoria = v_categoria;

  SET p_codice_specie = CONCAT(
    'SP-', v_tipo_fioritura, LPAD(v_nuovo_numero, 3, '0'), '-', v_tipo_ambiente
  );

```

```

INSERT INTO SpecieDiPiante (
    CodiceSpecie, NomeComune, NomeLatino, Tipologia, Esotica, Fiorita
) VALUES (
    p_codice_specie, p_nome_comune, p_nome_latino, p_tipologia, p_esotica, p_fiorita
);

IF p_fiorita = 1 THEN
    INSERT INTO SpecieFiorita (CodiceSpecie, Colorazioni)
    VALUES (p_codice_specie, p_colorazione);
END IF;

INSERT INTO StoricoPrezzi (
    DataVariazione, PrezzoPrecedente, PrezzoAttuale, SpeciePianta
) VALUES (
    NOW(6), NULL, p_prezzo_iniziale, p_codice_specie
);

COMMIT;
END

```

La procedura **InserisciSpecieConPrezzo** gestisce in maniera atomica l’inserimento di una nuova specie di pianta e del relativo prezzo iniziale. Il codice identificativo non viene fornito dall’utente, ma viene generato automaticamente all’interno della procedura sulla base della tipologia della specie e della caratteristica fiorita/non fiorita.

La procedura utilizza la tabella CodiceSpecieProgressivo per ottenere il successivo valore progressivo relativo alla categoria della specie. La riga del progressivo viene acquisita tramite FOR UPDATE, impedendo a esecuzioni concorrenti di generare lo stesso codice.

Dopo la generazione del codice viene inserita la nuova specie nella tabella SpecieDiPiante. Se la specie è fiorita viene inoltre inserita la relativa colorazione nella tabella SpecieFiorita.

Nella stessa transazione viene registrato il prezzo iniziale in StoricoPrezzi, utilizzando NOW(6) come istante della variazione. Per questa prima registrazione PrezzoPrecedente assume valore NULL, mentre PrezzoAttuale contiene il prezzo iniziale fornito dal responsabile commerciale.

L’intera operazione viene eseguita all’interno di un’unica transazione. In caso di errore viene eseguito il ROLLBACK e l’eccezione originaria viene propagata mediante RESIGNAL, evitando inserimenti parziali e garantendo che una nuova specie venga sempre registrata insieme al proprio prezzo iniziale.

### Procedura ModificaSpecie

```

CREATE DEFINER='root'@'localhost' PROCEDURE `ModificaSpecie`(
    IN p_codice_specie CHAR(10),
    IN p_nome_comune VARCHAR(45),
    IN p_nome_latino VARCHAR(45),
    IN p_tipologia ENUM('APPARTAMENTO', 'GIARDINO'),
    IN p_esotica TINYINT,
    IN p_fiorita TINYINT,
    IN p_colorazione ENUM('ROSSA', 'GIALLA', 'ARANCIONE', 'BLU', 'VIOLA', 'BIANCA',
'MULTICOLORE')

```

```
)
BEGIN
    DECLARE ordiniVendita INT DEFAULT 0;
    DECLARE ordiniRifornimento INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;

    IF NOT EXISTS (
        SELECT 1
        FROM SpecieDiPiante
        WHERE CodiceSpecie = p_codice_specie
        FOR UPDATE
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Nessuna specie trovata da modificare.';
    END IF;

    IF p_fiorita = 1 AND p_colorazione IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Colorazione obbligatoria per specie fiorita.';
    END IF;

    SELECT COUNT(*)
    INTO ordiniVendita
    FROM DettagliOrdine
    WHERE CodiceSpecie = p_codice_specie
    FOR UPDATE;

    SELECT COUNT(*)
    INTO ordiniRifornimento
    FROM DettaglioOrdineRifornimento
    WHERE CodiceSpecie = p_codice_specie
    FOR UPDATE;

    IF ordiniVendita > 0 OR ordiniRifornimento > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Impossibile modificare la specie presente in ordini di vendita o rifornimento esistenti.';
    END IF;

    UPDATE SpecieDiPiante
    SET NomeComune = p_nome_comune,
        NomeLatino = p_nome_latino,
        Tipologia = p_tipologia,
        Esotica = p_esotica,
        Fiorita = p_fiorita
    WHERE CodiceSpecie = p_codice_specie;
```

```
IF p_fiorita = 1 THEN
  IF EXISTS (
    SELECT 1
    FROM SpecieFiorita
    WHERE CodiceSpecie = p_codice_specie
  ) THEN
    UPDATE SpecieFiorita
    SET Colorazioni = p_colorazione
    WHERE CodiceSpecie = p_codice_specie;
  ELSE
    INSERT INTO SpecieFiorita (CodiceSpecie, Colorazioni)
    VALUES (p_codice_specie, p_colorazione);
  END IF;
ELSE
  DELETE FROM SpecieFiorita
  WHERE CodiceSpecie = p_codice_specie;
END IF;

COMMIT;

END
```

La procedura **ModificaSpecie** permette di aggiornare i dati di una specie già registrata. Viene avviata una transazione con livello di isolamento REPEATABLE READ per mantenere la stabilità delle letture durante tutta l'operazione. Per prima cosa la procedura controlla che la specie esista, in caso contrario viene interrotta con un errore. Successivamente verifica che la specie non sia coinvolta in ordini di vendita o di rifornimento già presenti, se così fosse, la modifica non è consentita, per evitare incoerenze nei dati.

La gestione delle eccezioni è centralizzata mediante un EXIT HANDLER che esegue il rollback della transazione e utilizza RESIGNAL per propagare l'errore originario. In questo modo vengono preservati i messaggi specifici generati dai controlli della procedura, evitando che errori differenti vengano convertiti in un unico messaggio generico.

## Procedura VisualizzaSpecie

```
DELIMITER //
CREATE PROCEDURE `VisualizzaSpecie`(
  IN codice_in CHAR(10)
)
BEGIN
SELECT
  s.CodiceSpecie,
  s.NomeComune,
  s.NomeLatino,
  s.Tipologia,
  s.Esotica,
  s.Fiorita,
sf.Colorazioni FROM
SpecieDiPiante s
  LEFT JOIN SpecieFiorita sf ON s.CodiceSpecie = sf.CodiceSpecie
WHERE s.CodiceSpecie = codice_in;
END //
DELIMITER ;
```

Questa procedura permette di consultare i dati completi di una singola specie, unendo le informazioni presenti in SpecieDiPiante con l'eventuale colorazione in SpecieFiorita. Trattandosi di un'operazione esclusivamente di lettura, non modifica alcun dato e quindi non richiede un livello di isolamento particolare oltre a quello predefinito di MySql.

## Procedura VisualizzaSpecieTotali

```
CREATE PROCEDURE `VisualizzaSpecieTotali`()
BEGIN
SELECT
  s.CodiceSpecie,
  s.NomeComune,
  s.NomeLatino,
  s.Tipologia,
  s.Esotica,
  s.Fiorita,
sf.Colorazioni FROM
SpecieDiPiante s
  LEFT JOIN SpecieFiorita sf ON s.CodiceSpecie = sf.CodiceSpecie
ORDER BY s.CodiceSpecie;
END //
```

La procedura restituisce l'elenco di tutte le specie registrate, comprensive di eventuali colorazioni, ordinando i risultati per codice. Anche in questo caso si tratta di una semplice consultazione, senza inserimenti o aggiornamenti, quindi non sono quindi necessari livelli di isolamento aggiuntivi.

### Procedura RicercaSpeciePerNome

```
DELIMITER //
CREATE PROCEDURE `RicercaSpeciePerNome`(IN termineRicerca VARCHAR(45))
BEGIN
    SELECT
        sp.CodiceSpecie,
        sp.NomeComune,
        sp.NomeLatino,
        sp.Tipologia,
        sp.Esotica,          sp.Fiorita,
        sf.Colorazioni
    FROM
        SpecieDiPiante sp
    LEFT JOIN
        SpecieFiorita sf ON sp.CodiceSpecie = sf.CodiceSpecie
WHERE
    sp.NomeComune LIKE CONCAT('%', termineRicerca, '%') OR
    sp.NomeLatino LIKE CONCAT('%', termineRicerca, '%');
END //
DELIMITER ;
```

Questa procedura consente di cercare le specie attraverso il nome comune o quello latino, utilizzando la clausola LIKE che permette ricerche parziali.

### Procedura RicercaSpeciePerTipologia

```
DELIMITER //
CREATE PROCEDURE `RicercaSpeciePerTipologia`(IN tipologiaRicerca VARCHAR(45))
BEGIN
    SELECT
        sp.CodiceSpecie,
        sp.NomeComune,
        sp.NomeLatino,
        sp.Tipologia,
        sp.Esotica,          sp.Fiorita,
        sf.Colorazioni
    FROM
        SpecieDiPiante sp
    LEFT JOIN
        SpecieFiorita sf ON sp.CodiceSpecie = sf.CodiceSpecie
WHERE
    sp.Tipologia = tipologiaRicerca;
END //
DELIMITER ;
```

Filtra le specie in base alla tipologia di appartenenza (“APPARTAMENTO” o “GIARDINO”), restituendo i dati principali e la colorazione associata se presente.

## Procedura RicercaSpeciePerColorazione

```
DELIMITER //
CREATE PROCEDURE `RicercaSpeciePerColorazione`(IN colorazioneRicerca VARCHAR(45))
BEGIN
SELECT
    sp.CodiceSpecie,
    sp.NomeComune,
    sp.NomeLatino,
    sp.Tipologia,
    sp.Esotica,          sp.Fiorita,
    sf.Colorazioni
FROM
    SpecieDiPiante sp
INNER JOIN
    SpecieFiorita sf ON sp.CodiceSpecie = sf.CodiceSpecie
WHERE
    sf.Colorazioni = colorazioneRicerca;
END //
DELIMITER ;
```

Questa procedura individua le specie in base al colore del fiore, sfruttando un INNER JOIN con la tabella SpecieFiorita.

## Procedura EliminaSpecie

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `EliminaSpecie`(IN pCodiceSpecie CHAR(10))
BEGIN
    DECLARE v_ordini_vendita_count INT DEFAULT 0;
    DECLARE v_ordini_rifornimento_count INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;

    IF NOT EXISTS (
        SELECT 1
        FROM SpecieDiPiante
        WHERE CodiceSpecie = pCodiceSpecie
        FOR UPDATE
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Nessuna specie trovata da rimuovere.';
    END IF;

    SELECT COUNT(*)
    INTO v_ordini_vendita_count
    FROM DettagliOrdine
    WHERE CodiceSpecie = pCodiceSpecie
    FOR UPDATE;

    SELECT COUNT(*)
    INTO v_ordini_rifornimento_count
    FROM DettagliOrdineRifornimento
    WHERE CodiceSpecie = pCodiceSpecie
```

```

FOR UPDATE;

IF v_ordini_vendita_count > 0 OR v_ordini_rifornimento_count > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Impossibile eliminare la specie presente in ordini di
vendita o rifornimento.';
END IF;

DELETE FROM SpecieDiPiante
WHERE CodiceSpecie = pCodiceSpecie;

COMMIT;
END

```

Questa procedura serve per eliminare una specie dal database. L'eliminazione è consentita solamente se la specie non è collegata ad alcun ordine di vendita o di rifornimento, per garantire la tracciabilità di essa e per evitare incoerenze nelle consultazioni degli ordini.

La procedura utilizza una transazione con livello di isolamento REPEATABLE READ, che assicura stabilità nella lettura e impedisce modifiche concorrenti durante i controlli. Le verifiche vengono eseguite bloccando con FOR UPDATE le righe interessate, se la specie è ancora associata a ordini, viene eseguito un ROLLBACK con messaggio di errore, mentre in assenza di vincoli la cancellazione viene completata e la transazione confermata.

### Procedura InserisciAziendaRivenditrice

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `InserisciAziendaRivenditrice`(
    IN pPartitaIVA CHAR(11),
    IN pNomeAzienda VARCHAR(100),
    IN pNomeReferente VARCHAR(50),
    IN pCognomeReferente VARCHAR(50),
    IN pViaLegale VARCHAR(100),
    IN pCapLegale CHAR(5),
    IN pCittaLegale VARCHAR(50),
    IN pViaFatturazione VARCHAR(100),
    IN pCapFatturazione CHAR(5),
    IN pCittaFatturazione VARCHAR(50),
    IN pContatti JSON
)
BEGIN
    DECLARE vIdReferente INT;
    DECLARE vIdIndirizzoLegale INT;
    DECLARE vIdIndirizzoFatturazione INT;
    DECLARE vIndex INT DEFAULT 0;
    DECLARE vCount INT DEFAULT 0;
    DECLARE vTipoContatto VARCHAR(20);
    DECLARE vValoreContatto VARCHAR(100);
    DECLARE vIdContatto INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;

```

```

IF pPartitaIVA IS NULL OR pPartitaIVA NOT REGEXP '^[0-9]{11}$' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Partita IVA non valida.';
END IF;

IF pNomeAzienda IS NULL OR CHAR_LENGTH(TRIM(pNomeAzienda)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nome azienda obbligatorio.';
END IF;

IF pNomeReferente IS NULL OR CHAR_LENGTH(TRIM(pNomeReferente)) = 0
    OR pCognomeReferente IS NULL OR CHAR_LENGTH(TRIM(pCognomeReferente)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Referente aziendale obbligatorio.';
END IF;

IF pViaLegale IS NULL OR CHAR_LENGTH(TRIM(pViaLegale)) = 0
    OR pCapLegale IS NULL OR pCapLegale NOT REGEXP '^[0-9]{5}$'
    OR pCittaLegale IS NULL OR CHAR_LENGTH(TRIM(pCittaLegale)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Indirizzo legale non valido.';
END IF;

IF pViaFatturazione IS NULL OR CHAR_LENGTH(TRIM(pViaFatturazione)) = 0
    OR pCapFatturazione IS NULL OR pCapFatturazione NOT REGEXP '^[0-9]{5}$'
    OR pCittaFatturazione IS NULL OR CHAR_LENGTH(TRIM(pCittaFatturazione)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Indirizzo di fatturazione non
valido.';
END IF;

IF pContatti IS NULL OR JSON_TYPE(pContatti) <> 'ARRAY' OR JSON_LENGTH(pContatti) = 0
THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Aggiungere almeno un contatto
aziendale.';
END IF;

IF EXISTS (SELECT 1 FROM AziendeRivenditrici WHERE PartitaIVA = pPartitaIVA) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Partita IVA già presente.';
END IF;

INSERT INTO ReferentiAziendali (Nome, Cognome)
VALUES (TRIM(pNomeReferente), TRIM(pCognomeReferente));
SET vIdReferente = LAST_INSERT_ID();

INSERT INTO Indirizzi (TipoIndirizzo, Via, CAP, Citta)
VALUES ('LEGALE', TRIM(pViaLegale), pCapLegale, TRIM(pCittaLegale));
SET vIdIndirizzoLegale = LAST_INSERT_ID();

INSERT INTO Indirizzi (TipoIndirizzo, Via, CAP, Citta)
VALUES ('FATTURAZIONE', TRIM(pViaFatturazione), pCapFatturazione,
TRIM(pCittaFatturazione));
SET vIdIndirizzoFatturazione = LAST_INSERT_ID();

INSERT INTO AziendeRivenditrici (PartitaIVA, NomeAzienda, IdIndirizzoLegale,
IdIndirizzoFatturazione, IdReferente)
VALUES (pPartitaIVA, TRIM(pNomeAzienda), vIdIndirizzoLegale, vIdIndirizzoFatturazione,
vIdReferente);

SET vCount = JSON_LENGTH(pContatti);
WHILE vIndex < vCount DO
    SET vTipoContatto = UPPER(TRIM(JSON_UNQUOTE(JSON_EXTRACT(pContatti, CONCAT('$[',
vIndex, '].type')))));
    SET vValoreContatto = TRIM(JSON_UNQUOTE(JSON_EXTRACT(pContatti, CONCAT('$[',
vIndex, '].value'))));

    IF vTipoContatto IS NULL OR vTipoContatto NOT IN ('TELEFONO', 'CELLULARE',
'EMAIL') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tipo contatto non valido.';

```

```

        END IF;
        IF vValoreContatto IS NULL OR CHAR_LENGTH(vValoreContatto) = 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Valore contatto obbligatorio.';
        END IF;

        INSERT IGNORE INTO Contatti (TipoContatto, ValoreContatto) VALUES (vTipoContatto,
vValoreContatto);
        SELECT IdContatto INTO vIdContatto FROM Contatti
        WHERE TipoContatto = vTipoContatto AND ValoreContatto = vValoreContatto LIMIT 1;
        INSERT INTO PossiedeRecapito (PartitaIVA, IdContatto) VALUES (pPartitaIVA,
vIdContatto);
        SET vIndex = vIndex + 1;
    END WHILE;

    COMMIT;
END

```

La procedura permette di registrare atomicamente una nuova azienda rivenditrice insieme al referente, agli indirizzi legale e di fatturazione e ai relativi contatti. I contatti vengono ricevuti tramite un array JSON, validati e registrati nella tabella CONTATTI; successivamente vengono associati all'azienda mediante POSSIEDERECAPIITO. La procedura verifica inoltre la validità dei dati obbligatori e richiede la presenza di almeno un contatto. Tutte le operazioni vengono eseguite all'interno della stessa transazione, con ROLLBACK in caso di errore.

### Procedura ModificaAziendaRivenditrice

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `ModificaAziendaRivenditrice`(
    IN p_partitaIVA CHAR(11),
    IN p_nomeAzienda VARCHAR(100),
    IN p_nomeReferente VARCHAR(50),
    IN p_cognomeReferente VARCHAR(50),
    IN p_viaLegale VARCHAR(100),
    IN p_capLegale CHAR(5),
    IN p_cittaLegale VARCHAR(50),
    IN p_viaFatturazione VARCHAR(100),
    IN p_capFatturazione CHAR(5),
    IN p_cittaFatturazione VARCHAR(50),
    IN p_contatti JSON
)
BEGIN
    DECLARE idLegale INT;
    DECLARE idFatt INT;
    DECLARE idRef INT;
    DECLARE nuovoIdLegale INT;
    DECLARE nuovoIdFatt INT;
    DECLARE nuovoIdRef INT;
    DECLARE vIndex INT DEFAULT 0;
    DECLARE vCount INT DEFAULT 0;
    DECLARE vTipoContatto VARCHAR(20);
    DECLARE vValoreContatto VARCHAR(100);
    DECLARE vIdContatto INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

```

```

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

SELECT IdIndirizzoLegale, IdIndirizzoFatturazione, IdReferente
INTO idLegale, idFatt, idRef
FROM AziendeRivenditrici
WHERE PartitaIVA = p_partitaIVA
FOR UPDATE;

IF idLegale IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Azienda non trovata.';
END IF;

IF p_nomeAzienda IS NULL OR CHAR_LENGTH(TRIM(p_nomeAzienda)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nome azienda obbligatorio.';
END IF;

IF p_nomeReferente IS NULL OR CHAR_LENGTH(TRIM(p_nomeReferente)) = 0
OR p_cognomeReferente IS NULL OR CHAR_LENGTH(TRIM(p_cognomeReferente)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Referente aziendale obbligatorio.';
END IF;

IF p_viaLegale IS NULL OR CHAR_LENGTH(TRIM(p_viaLegale)) = 0
OR p_capLegale IS NULL OR p_capLegale NOT REGEXP '^[0-9]{5}$'
OR p_cittaLegale IS NULL OR CHAR_LENGTH(TRIM(p_cittaLegale)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Indirizzo legale non valido.';
END IF;

IF p_viaFatturazione IS NULL OR CHAR_LENGTH(TRIM(p_viaFatturazione)) = 0
OR p_capFatturazione IS NULL OR p_capFatturazione NOT REGEXP '^[0-9]{5}$'
OR p_cittaFatturazione IS NULL OR CHAR_LENGTH(TRIM(p_cittaFatturazione)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Indirizzo di fatturazione non
valido.';
END IF;

IF p_contatti IS NULL OR JSON_TYPE(p_contatti) <> 'ARRAY' OR JSON_LENGTH(p_contatti) =
0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Aggiungere almeno un contatto
aziendale.';
END IF;

INSERT INTO ReferentiAziendali (Nome, Cognome)
VALUES (TRIM(p_nomeReferente), TRIM(p_cognomeReferente));
SET nuovoIdRef = LAST_INSERT_ID();

INSERT INTO Indirizzi (TipoIndirizzo, Via, CAP, Citta)
VALUES ('LEGALE', TRIM(p_viaLegale), p_capLegale, TRIM(p_cittaLegale));
SET nuovoIdLegale = LAST_INSERT_ID();

INSERT INTO Indirizzi (TipoIndirizzo, Via, CAP, Citta)
VALUES ('FATTURAZIONE', TRIM(p_viaFatturazione), p_capFatturazione,
TRIM(p_cittaFatturazione));
SET nuovoIdFatt = LAST_INSERT_ID();

UPDATE AziendeRivenditrici
SET NomeAzienda = TRIM(p_nomeAzienda),
    IdIndirizzoLegale = nuovoIdLegale,
    IdIndirizzoFatturazione = nuovoIdFatt,
    IdReferente = nuovoIdRef
WHERE PartitaIVA = p_partitaIVA;

DELETE FROM PossiedeRecapito WHERE PartitaIVA = p_partitaIVA;

SET vCount = JSON_LENGTH(p_contatti);
WHILE vIndex < vCount DO
    SET vTipoContatto = UPPER(TRIM(JSON_UNQUOTE(JSON_EXTRACT(p_contatti, CONCAT('$[',
vIndex, '].type')))));

```

```

        SET vValoreContatto = TRIM(JSON_UNQUOTE(JSON_EXTRACT(p_contatti, CONCAT('$[',
vIndex, '].value'))));
        IF vTipoContatto IS NULL OR vTipoContatto NOT IN ('TELEFONO', 'CELLULARE',
'EMAIL') THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tipo contatto non valido.';
        END IF;
        IF vValoreContatto IS NULL OR CHAR_LENGTH(vValoreContatto) = 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Valore contatto obbligatorio.';
        END IF;
        INSERT IGNORE INTO Contatti (TipoContatto, ValoreContatto) VALUES (vTipoContatto,
vValoreContatto);
        SELECT IdContatto INTO vIdContatto FROM Contatti
        WHERE TipoContatto = vTipoContatto AND ValoreContatto = vValoreContatto LIMIT 1;
        INSERT INTO PossiedeRecapito (PartitaIVA, IdContatto) VALUES (p_partitaIVA,
vIdContatto);
        SET vIndex = vIndex + 1;
    END WHILE;

    DELETE FROM Indirizzi
    WHERE IdIndirizzo = idLegale
        AND NOT EXISTS (SELECT 1 FROM AziendeRivenditrici a WHERE a.IdIndirizzoLegale =
idLegale OR a.IdIndirizzoFatturazione = idLegale)
        AND NOT EXISTS (SELECT 1 FROM IndirizzoFornitore f WHERE f.IdIndirizzo = idLegale);

    IF idFatt IS NOT NULL AND idFatt <> idLegale THEN
        DELETE FROM Indirizzi
        WHERE IdIndirizzo = idFatt
            AND NOT EXISTS (SELECT 1 FROM AziendeRivenditrici a WHERE a.IdIndirizzoLegale =
idFatt OR a.IdIndirizzoFatturazione = idFatt)
            AND NOT EXISTS (SELECT 1 FROM IndirizzoFornitore f WHERE f.IdIndirizzo =
idFatt);
    END IF;

    DELETE FROM ReferentiAziendali
    WHERE IdReferente = idRef
        AND NOT EXISTS (SELECT 1 FROM AziendeRivenditrici a WHERE a.IdReferente = idRef);

    COMMIT;
END

```

La procedura **ModificaAziendaRivenditrice** aggiorna atomicamente i dati di un'azienda rivenditrice, compresi referente, indirizzi e contatti. I nuovi dati vengono validati e registrati, quindi l'azienda viene associata ai nuovi indirizzi e al nuovo referente. I recapiti precedentemente associati vengono sostituiti con quelli ricevuti tramite array JSON. Al termine, i vecchi indirizzi e il vecchio referente vengono eliminati solo se non risultano più utilizzati da altre entità. L'intera operazione viene eseguita in una transazione con isolamento REPEATABLE READ e viene annullata mediante ROLLBACK in caso di errore.

**Procedura EliminaAziendaRivenditrice**

```
CREATE DEFINER='root'@'localhost' PROCEDURE `EliminaAziendaRivenditrice`( IN p_partitaIVA
CHAR(11) )
BEGIN
    DECLARE idLegale INT;
    DECLARE idFatt INT;
    DECLARE idRef INT;
    DECLARE ordiniAssociati INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
    START TRANSACTION;

    SELECT IdIndirizzoLegale, IdIndirizzoFatturazione, IdReferente
    INTO idLegale, idFatt, idRef
    FROM AziendeRivenditrici
    WHERE PartitaIVA = p_partitaIVA
    FOR UPDATE;

    IF idLegale IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Azienda non trovata.';
    END IF;

    SELECT COUNT(*) INTO ordiniAssociati
    FROM OrdiniDiVendita
    WHERE PartitaIVA = p_partitaIVA
    FOR UPDATE;

    IF ordiniAssociati > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Impossibile eliminare l azienda:
esistono ordini di vendita associati.';
    END IF;

    DELETE FROM PossiedeRecapito WHERE PartitaIVA = p_partitaIVA;
    DELETE FROM AziendeRivenditrici WHERE PartitaIVA = p_partitaIVA;

    DELETE FROM Indirizzi
    WHERE IdIndirizzo = idLegale
    AND NOT EXISTS (SELECT 1 FROM AziendeRivenditrici a WHERE a.IdIndirizzoLegale =
idLegale OR a.IdIndirizzoFatturazione = idLegale)
    AND NOT EXISTS (SELECT 1 FROM IndirizzoFornitore f WHERE f.IdIndirizzo = idLegale);

    IF idFatt IS NOT NULL AND idFatt <> idLegale THEN
        DELETE FROM Indirizzi
        WHERE IdIndirizzo = idFatt
        AND NOT EXISTS (SELECT 1 FROM AziendeRivenditrici a WHERE a.IdIndirizzoLegale =
idFatt OR a.IdIndirizzoFatturazione = idFatt)
        AND NOT EXISTS (SELECT 1 FROM IndirizzoFornitore f WHERE f.IdIndirizzo =
idFatt);
    END IF;

    DELETE FROM ReferentiAziendali
    WHERE IdReferente = idRef
    AND NOT EXISTS (SELECT 1 FROM AziendeRivenditrici a WHERE a.IdReferente = idRef);

    COMMIT;
END
```

La procedura elimina un'azienda rivenditrice solo se non risultano presenti ordini di vendita associati. Dopo aver rimosso le associazioni con i recapiti e la riga dell'azienda, elimina gli indirizzi e il referente collegati esclusivamente se non risultano più utilizzati da altre entità. L'operazione viene eseguita all'interno di una transazione con isolamento REPEATABLE READ; in caso di errore viene effettuato il ROLLBACK.

### Procedura VisualizzaAziendeClienti

```
DELIMITER //
CREATE PROCEDURE `VisualizzaAziendeClienti`()
BEGIN
  SELECT
    a.PartitaIVA,
    a.NomeAzienda,      il.Via AS ViaLegale,      il.CAP AS CAPLegale,      il.Citta AS
CittaLegale,      ifa.Via AS ViaFatturazione,      ifa.CAP AS CAPFatturazione,
    ifa.Citta AS CittaFatturazione,
    r.Nome AS NomeReferente,
    r.Cognome AS CognomeReferente
  FROM AziendeRivenditrici a
  JOIN Indirizzi il ON a.IdIndirizzoLegale = il.IdIndirizzo
  JOIN Indirizzi ifa ON a.IdIndirizzoFatturazione = ifa.IdIndirizzo
  JOIN ReferentiAziendali r ON a.IdReferente = r.IdReferente;
END //
```

Questa procedura restituisce l'elenco delle aziende rivenditrici completo di indirizzi e referente, tramite join tra AziendeRivenditrici, Indirizzi e ReferentiAziendali. È una procedura di sola lettura non richiede livelli di isolamento oltre al default di MySQL.

### Procedura RicercaAziendaPerPartitaIVA

```
DELIMITER //
CREATE PROCEDURE `RicercaAziendaPerPartitaIVA`(
  IN p_partitaIVA CHAR(11)
)
BEGIN
  SELECT
    ar.PartitaIVA,
    ar.NomeAzienda,
    r.Nome AS NomeReferente,
    r.Cognome AS CognomeReferente,
    il.Via AS ViaLegale,      il.CAP AS
CapLegale,      il.Citta AS
CittaLegale,      ifat.Via AS
ViaFatturazione,      ifat.CAP AS
CapFatturazione,      ifat.Citta AS
CittaFatturazione
  FROM AziendeRivenditrici ar
  JOIN ReferentiAziendali r ON ar.IdReferente = r.IdReferente
  JOIN Indirizzi il ON ar.IdIndirizzoLegale = il.IdIndirizzo
  JOIN Indirizzi ifat ON ar.IdIndirizzoFatturazione = ifat.IdIndirizzo
  WHERE ar.PartitaIVA = p_partitaIVA;
END //
DELIMITER ;
```

Questa procedura consente di ottenere la scheda completa di una singola azienda a partire dalla Partita IVA.

### Procedura VisualizzaContattiAzienda

```
CREATE PROCEDURE VisualizzaContattiAzienda(  
    IN p_PartitaIVA CHAR(11)  
)  
BEGIN  
    SELECT c.TipoContatto, c.ValoreContatto  
    FROM Contatti c  
    INNER JOIN PossiedeRecapito pr  
        ON c.IdContatto = pr.IdContatto  
    WHERE pr.PartitaIVA = p_PartitaIVA  
    ORDER BY c.IdContatto;  
END;
```

La procedura **VisualizzaContattiAzienda** restituisce tutti i recapiti associati a una specifica azienda rivenditrice, individuata tramite Partita IVA. I dati vengono recuperati mediante la relazione PossiedeRecapito, che collega l'azienda ai relativi record della tabella Contatti.

### Procedura InserisciFornitoreCompleto

```
CREATE DEFINER='root'@'localhost' PROCEDURE `InserisciFornitoreCompleto`(  
    IN p_Nome VARCHAR(100),  
    IN p_CodiceFiscale VARCHAR(16),  
    IN p_Indirizzi JSON,  
    OUT p_CodiceFornitore INT  
)  
BEGIN  
    DECLARE v_index INT DEFAULT 0;  
    DECLARE v_count INT DEFAULT 0;  
    DECLARE v_tipo VARCHAR(20);  
    DECLARE v_via VARCHAR(100);  
    DECLARE v_cap VARCHAR(5);  
    DECLARE v_citta VARCHAR(50);  
    DECLARE v_idIndirizzo INT;  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;
```

```
IF p_Nome IS NULL OR CHAR_LENGTH(TRIM(p_Nome)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nome fornitore obbligatorio.';
END IF;

IF p_CodiceFiscale IS NULL OR CHAR_LENGTH(TRIM(p_CodiceFiscale)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fiscale obbligatorio.';
END IF;

IF p_Indirizzi IS NULL OR JSON_TYPE(p_Indirizzi) <> 'ARRAY' OR
JSON_LENGTH(p_Indirizzi) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Un fornitore deve avere almeno un
indirizzo.';
END IF;

IF EXISTS (SELECT 1 FROM Fornitore WHERE CodiceFiscale = p_CodiceFiscale) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fiscale già presente.';
END IF;

INSERT INTO Fornitore(Nome, CodiceFiscale)
VALUES (TRIM(p_Nome), UPPER(TRIM(p_CodiceFiscale)));

SET p_CodiceFornitore = LAST_INSERT_ID();
SET v_count = JSON_LENGTH(p_Indirizzi);

WHILE v_index < v_count DO
    SET v_tipo = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
    '].type')));
    SET v_via = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
    '].street')));
    SET v_cap = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
    '].postalCode')));
    SET v_citta = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
    '].city')));

    IF v_tipo IS NULL OR v_tipo NOT IN ('LEGALE', 'FATTURAZIONE') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tipo indirizzo non valido.';
    END IF;
```

```
IF v_via IS NULL OR CHAR_LENGTH(TRIM(v_via)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Via indirizzo obbligatoria.';
END IF;

IF v_cap IS NULL OR v_cap NOT REGEXP '^[0-9]{5}$' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CAP indirizzo non valido.';
END IF;

IF v_citta IS NULL OR CHAR_LENGTH(TRIM(v_citta)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Città indirizzo obbligatoria.';
END IF;

INSERT INTO Indirizzi(TipoIndirizzo, Via, CAP, Citta)
VALUES (v_tipo, TRIM(v_via), v_cap, TRIM(v_citta));

SET v_idIndirizzo = LAST_INSERT_ID();

INSERT INTO IndirizzoFornitore(CodiceFornitore, IdIndirizzo, Tipo)
VALUES (p_CodiceFornitore, v_idIndirizzo, v_tipo);

SET v_index = v_index + 1;
END WHILE;

COMMIT;

END
```

La procedura `InserisciFornitoreCompleto` gestisce atomicamente l’inserimento di un nuovo fornitore e dei relativi indirizzi. Riceve il nome, il codice fiscale e un array JSON contenente gli indirizzi da registrare.

Prima dell’inserimento vengono verificati la validità dei dati obbligatori, la presenza di almeno un indirizzo e l’univocità del codice fiscale. Dopo la creazione del fornitore viene recuperato il codice generato mediante `LAST_INSERT_ID()`.

Gli indirizzi presenti nell’array JSON vengono quindi elaborati uno alla volta, validati e inseriti nella tabella `Indirizzi`; successivamente viene registrata l’associazione con il fornitore mediante `IndirizzoFornitore`.

L’intera operazione viene eseguita all’interno di una singola transazione. In caso di errore viene effettuato il `ROLLBACK` e l’errore originario viene propagato mediante `RESIGNAL`, evitando la presenza di fornitori registrati solo parzialmente.

**Procedura ModificaFornitoreCompleto**

```

CREATE DEFINER='root'@'localhost' PROCEDURE `ModificaFornitoreCompleto`(
  IN p_CodiceFornitore INT,
  IN p_Nome VARCHAR(100),
  IN p_CodiceFiscale VARCHAR(16),
  IN p_Indirizzi JSON
)
BEGIN
  DECLARE v_index INT DEFAULT 0;
  DECLARE v_count INT DEFAULT 0;
  DECLARE v_tipo VARCHAR(20);
  DECLARE v_via VARCHAR(100);
  DECLARE v_cap VARCHAR(5);
  DECLARE v_citta VARCHAR(50);
  DECLARE v_idIndirizzo INT;
  DECLARE v_ordiniAssociati INT DEFAULT 0;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  START TRANSACTION;

  IF p_CodiceFornitore IS NULL OR p_CodiceFornitore <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fornitore non valido.';
  END IF;

  IF p_Nome IS NULL OR CHAR_LENGTH(TRIM(p_Nome)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nome fornitore obbligatorio.';
  END IF;

  IF p_CodiceFiscale IS NULL OR CHAR_LENGTH(TRIM(p_CodiceFiscale)) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fiscale obbligatorio.';
  END IF;

  IF p_Indirizzi IS NULL OR JSON_TYPE(p_Indirizzi) <> 'ARRAY' OR
  JSON_LENGTH(p_Indirizzi) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Un fornitore deve avere almeno un
indirizzo.';
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM Fornitore
    WHERE CodiceFornitore = p_CodiceFornitore
    FOR UPDATE
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fornitore non trovato.';
  END IF;

  SELECT COUNT(*)
    INTO v_ordiniAssociati
  FROM OrdineRifornimento
  WHERE CodiceFornitore = p_CodiceFornitore
  FOR UPDATE;

  IF v_ordiniAssociati > 0 THEN
    SIGNAL SQLSTATE '45000'

```

```

        SET MESSAGE_TEXT = 'Impossibile modificare il fornitore ha ordini di rifornimento
associati.';
    END IF;

    IF EXISTS (
        SELECT 1 FROM Fornitore
        WHERE CodiceFiscale = p_CodiceFiscale
        AND CodiceFornitore <> p_CodiceFornitore
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fiscale già esistente per un
altro fornitore.';
    END IF;

    UPDATE Fornitore
        SET Nome = TRIM(p_Nome), CodiceFiscale = UPPER(TRIM(p_CodiceFiscale))
        WHERE CodiceFornitore = p_CodiceFornitore;

    DELETE i
    FROM Indirizzi i
    INNER JOIN IndirizzoFornitore target
        ON target.IdIndirizzo = i.IdIndirizzo
        AND target.CodiceFornitore = p_CodiceFornitore
    WHERE NOT EXISTS (
        SELECT 1
        FROM IndirizzoFornitore other
        WHERE other.IdIndirizzo = i.IdIndirizzo
        AND other.CodiceFornitore <> p_CodiceFornitore
    )
    AND NOT EXISTS (
        SELECT 1
        FROM AziendeRivenditrici a
        WHERE a.IdIndirizzoLegale = i.IdIndirizzo
        OR a.IdIndirizzoFatturazione = i.IdIndirizzo
    );

    DELETE FROM IndirizzoFornitore
    WHERE CodiceFornitore = p_CodiceFornitore;

    SET v_count = JSON_LENGTH(p_Indirizzi);

    WHILE v_index < v_count DO
        SET v_tipo = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
        '].type')));
        SET v_via = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
        '].street')));
        SET v_cap = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
        '].postalCode')));
        SET v_citta = JSON_UNQUOTE(JSON_EXTRACT(p_Indirizzi, CONCAT('$[', v_index,
        '].city')));

        IF v_tipo IS NULL OR v_tipo NOT IN ('LEGALE', 'FATTURAZIONE') THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Tipo indirizzo non valido.';
        END IF;

        IF v_via IS NULL OR CHAR_LENGTH(TRIM(v_via)) = 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Via indirizzo obbligatoria.';
        END IF;

        IF v_cap IS NULL OR v_cap NOT REGEXP '^[0-9]{5}$' THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'CAP indirizzo non valido.';
        END IF;

        IF v_citta IS NULL OR CHAR_LENGTH(TRIM(v_citta)) = 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Città indirizzo obbligatoria.';

```

```
END IF;

INSERT INTO Indirizzi(TipoIndirizzo, Via, CAP, Citta)
VALUES (v_tipo, TRIM(v_via), v_cap, TRIM(v_citta));

SET v_idIndirizzo = LAST_INSERT_ID();

INSERT INTO IndirizzoFornitore(CodiceFornitore, IdIndirizzo, Tipo)
VALUES (p_CodiceFornitore, v_idIndirizzo, v_tipo);

SET v_index = v_index + 1;
END WHILE;

COMMIT;
END
```

La procedura **ModificaFornitoreCompleto** consente di modificare in maniera atomica i dati anagrafici e gli indirizzi di un fornitore esistente.

La procedura verifica la validità del codice fornitore, dei dati obbligatori e degli indirizzi ricevuti. Viene inoltre controllata l'esistenza del fornitore e l'assenza di ordini di rifornimento associati, poiché in presenza di ordini storici la modifica non è consentita.

Dopo l'aggiornamento dei dati anagrafici, le precedenti associazioni con gli indirizzi vengono rimosse e sostituite con quelle ricevute nell'array JSON. L'eliminazione degli indirizzi tiene conto di eventuali riferimenti da parte di altre entità, evitando la rimozione di dati ancora utilizzati.

L'operazione è racchiusa in un'unica transazione e utilizza ROLLBACK e RESIGNAL in caso di errore.

### Procedura RicercaFornitorePerCodice

```
CREATE PROCEDURE RicercaFornitorePerCodice(
    IN p_CodiceFornitore INT
)
BEGIN
    SELECT CodiceFornitore, Nome, CodiceFiscale
    FROM Fornitore
    WHERE CodiceFornitore = p_CodiceFornitore;
END;
```

La procedura **RicercaFornitorePerCodice** recupera i dati anagrafici di uno specifico fornitore tramite il relativo codice identificativo, restituendo codice fornitore, nome e codice fiscale.

### Procedura VisualizzaIndirizziFornitore

```
CREATE PROCEDURE VisualizzaIndirizziFornitore(  
    IN p_CodiceFornitore INT  
)  
BEGIN  
    SELECT i.IdIndirizzo, i.Via, i.Citta, i.CAP, inf.Tipo  
    FROM IndirizzoFornitore inf  
    INNER JOIN Indirizzi i  
        ON inf.IdIndirizzo = i.IdIndirizzo  
    WHERE inf.CodiceFornitore = p_CodiceFornitore  
    ORDER BY CASE inf.Tipo  
        WHEN 'LEGALE' THEN 1  
        WHEN 'FATTURAZIONE' THEN 2  
        ELSE 3  
    END;  
END;
```

La procedura **VisualizzaIndirizziFornitore** restituisce tutti gli indirizzi associati a uno specifico fornitore tramite la relazione *IndirizzoFornitore*. Per ciascun indirizzo vengono mostrati i dati geografici e la relativa tipologia; i risultati vengono ordinati privilegiando l'indirizzo legale e successivamente quello di fatturazione.

### Procedura VisualizzaFornitori

```
CREATE PROCEDURE VisualizzaFornitori()  
BEGIN  
    SELECT CodiceFornitore, Nome, CodiceFiscale  
    FROM Fornitore  
    ORDER BY Nome;  
END;
```

La procedura **VisualizzaFornitori** restituisce l'elenco completo dei fornitori registrati nel sistema, mostrando per ciascuno il codice identificativo, il nome e il codice fiscale. I risultati vengono ordinati alfabeticamente per nome.

**Procedura EliminaFornitore**

```
CREATE DEFINER='root'@'localhost' PROCEDURE `EliminaFornitore`(  
    IN p_CodiceFornitore INT )  
BEGIN  
    DECLARE ordiniAssociati INT DEFAULT 0;  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
    START TRANSACTION;  
  
    IF NOT EXISTS (  
        SELECT 1 FROM Fornitore  
        WHERE CodiceFornitore = p_CodiceFornitore  
        FOR UPDATE  
    ) THEN  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fornitore non trovato.';  
    END IF;  
  
    SELECT COUNT(*)  
        INTO ordiniAssociati  
    FROM OrdineRifornimento  
    WHERE CodiceFornitore = p_CodiceFornitore  
    FOR UPDATE;  
  
    IF ordiniAssociati > 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Impossibile eliminare il fornitore ha ordini di rifornimento  
associati.';  
    END IF;  
  
    DELETE i  
    FROM Indirizzi i  
    INNER JOIN IndirizzoFornitore target  
        ON target.IdIndirizzo = i.IdIndirizzo  
        AND target.CodiceFornitore = p_CodiceFornitore  
    WHERE NOT EXISTS (  
        SELECT 1  
        FROM IndirizzoFornitore other  
        WHERE other.IdIndirizzo = i.IdIndirizzo  
            AND other.CodiceFornitore <> p_CodiceFornitore  
    )  
    AND NOT EXISTS (  
        SELECT 1  
        FROM AziendeRivenditrici a  
        WHERE a.IdIndirizzoLegale = i.IdIndirizzo  
            OR a.IdIndirizzoFatturazione = i.IdIndirizzo  
    );  
  
    DELETE FROM IndirizzoFornitore  
    WHERE CodiceFornitore = p_CodiceFornitore;  
  
    DELETE FROM Fornitore WHERE CodiceFornitore = p_CodiceFornitore;  
  
    COMMIT;  
END
```

La procedura consente di eliminare un fornitore solo se non risultano presenti ordini di rifornimento associati. Prima della cancellazione vengono gestiti gli indirizzi collegati: sono eliminati solo quelli che non risultano utilizzati da altri fornitori o da aziende rivenditrici. Successivamente vengono rimosse le associazioni in IndirizzoFornitore e infine il fornitore. L'intera operazione è eseguita all'interno di una transazione con isolamento REPEATABLE READ.

### Procedura InserisciFornituraSpecie

```
DELIMITER //
CREATE PROCEDURE `InserisciFornituraSpecie`(
    IN p_CodiceFornitore INT,
    IN p_CodiceSpecie CHAR(10)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;

    IF NOT EXISTS (
        SELECT 1 FROM Fornitore
        WHERE CodiceFornitore = p_CodiceFornitore
        FOR UPDATE
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fornitore non trovato.';
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM SpecieDiPiante
        WHERE CodiceSpecie = p_CodiceSpecie
        FOR UPDATE
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Specie non trovata.';
    END IF;

    INSERT INTO FornituraSpecie (CodiceFornitore, CodiceSpecie, Quantita, Attiva)
    VALUES (p_CodiceFornitore, p_CodiceSpecie, 0, TRUE)
    ON DUPLICATE KEY UPDATE Attiva = TRUE;

    COMMIT;
END //
DELIMITER ;
```

Questa procedura serve a collegare una specie a un fornitore, registrandola come fornitura attiva. Prima di procedere verifica l'esistenza sia del fornitore sia della specie, bloccando le relative righe con FOR UPDATE per evitare modifiche concorrenti. Se il collegamento non esiste, inserisce una nuova riga in FornituraSpecie con quantità iniziale pari a 0 e stato Attiva = TRUE, qualora invece il collegamento è già presente, lo riattiva tramite ON DUPLICATE KEY UPDATE. L'intera operazione è gestita in transazione con isolamento READ COMMITTED, che previene letture sporche.

**Procedura EliminaFornituraSpecie**

```
DELIMITER //
CREATE PROCEDURE `EliminaFornituraSpecie`(
  IN p_CodiceFornitore INT,
  IN p_CodiceSpecie CHAR(10)
)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  UPDATE FornituraSpecie
  SET Attiva = FALSE
  WHERE CodiceFornitore = p_CodiceFornitore
  AND CodiceSpecie = p_CodiceSpecie;

  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Nessuna fornitura trovata.';
  END IF;

  COMMIT;
END // DELIMITER ;
```

Questa procedura non cancella effettivamente una fornitura, ma la disattiva impostando il campo `Attiva = FALSE` per la coppia fornitore e specie indicata, così da mantenere lo storico dei dati. Dopo l'aggiornamento controlla il valore di `ROW_COUNT()` per verificare che la fornitura esistesse davvero, se non viene trovata alcuna corrispondenza, segnala un errore. L'operazione è gestita con la transazione con isolamento `READ COMMITTED`. In caso di errore viene eseguito un `ROLLBACK`.

**Procedura SpecieForniteDaFornitore**

```
DELIMITER //
CREATE PROCEDURE `SpecieForniteDaFornitore`(
    IN p_CodiceFornitore INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        RESIGNAL;
    END;
    IF p_CodiceFornitore IS NULL OR p_CodiceFornitore <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fornitore non valido';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM Fornitore WHERE CodiceFornitore = p_CodiceFornitore) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fornitore non trovato';
    END IF;

    SELECT
        s.CodiceSpecie,
        s.NomeLatino,
        s.NomeComune,
        s.Tipologia,

        s.Esotica,
        sf.Colorazioni AS Colorazione
    FROM FornituraSpecie fs
    INNER JOIN SpecieDiPiante s ON fs.CodiceSpecie = s.CodiceSpecie
    LEFT JOIN SpecieFiorita sf ON s.CodiceSpecie = sf.CodiceSpecie
    WHERE fs.CodiceFornitore = p_CodiceFornitore
        AND fs.Attiva = 1
    ORDER BY s.NomeComune, s.CodiceSpecie;
END // DELIMITER ;
```

Questa procedura restituisce l'elenco delle specie attive fornite da un determinato fornitore. Prima esegue controlli sull'input e sull'esistenza del fornitore, poi recupera i dati tramite join tra FornituraSpecie, SpecieDiPiante e SpecieFiorita.

## Procedura InserisciGiacenza

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `InserisciGiacenza` (  
    IN var_CodiceSpecie CHAR(10),  
    IN var_Quantita INT  
)  
BEGIN  
    DECLARE EXIT HANDLER FOR 1062  
    BEGIN  
        ROLLBACK;  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Questa specie ha gia una giacenza, può essere solo  
modificata.';  
    END;  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    IF var_Quantita < 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'La quantità di magazzino non può essere negativa.';  
    END IF;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    INSERT INTO Magazzino (CodiceSpecie, Quantita)  
    VALUES (var_CodiceSpecie, var_Quantita);  
  
    COMMIT;  
END
```

Questa procedura registra la giacenza iniziale di una specie in magazzino. Se esiste già una riga per quella specie, l'handler intercetta l'errore e blocca l'operazione, invitando il responsabile logistico a utilizzare la modifica della giacenza invece dell'inserimento.

La transazione utilizza come livello di isolamento READ COMMITTED che è sufficiente per un inserimento evitando di introdurre lock.

**Procedura ModificaGiacenza**

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ModificaGiacenza`(  
    IN var_CodiceSpecie CHAR(10),  
    IN var_NuovaQuantita INT  
)  
BEGIN  
    DECLARE v_exist INT DEFAULT 0;  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    IF var_NuovaQuantita < 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'La quantità di magazzino non può essere negativa.';  
    END IF;  
  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
    START TRANSACTION;  
  
    SELECT COUNT(*)  
    INTO v_exist  
    FROM Magazzino  
    WHERE CodiceSpecie = var_CodiceSpecie  
    FOR UPDATE;  
  
    IF v_exist = 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Specie non trovata nelle giacenze.';  
    END IF;  
  
    UPDATE Magazzino  
    SET Quantita = var_NuovaQuantita  
    WHERE CodiceSpecie = var_CodiceSpecie;  
  
    COMMIT;  
END
```

La procedura aggiorna la quantità in magazzino per una specie esistente. Prima dell'aggiornamento verifica che la nuova quantità non sia negativa; successivamente controlla l'esistenza della riga in Magazzino e la blocca mediante FOR UPDATE. L'operazione viene eseguita in una transazione con isolamento REPEATABLE READ. In caso di errore viene effettuato il ROLLBACK e l'errore originario viene propagato mediante RESIGNAL.

**Procedura InserisciOrdineRifornimentoCompleto**

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `InserisciOrdineRifornimentoCompleto`(
  IN p_codiceFornitore INT,
  IN p_dataOrdine DATE,
  IN p_emailContatto VARCHAR(100),
  IN p_viaConsegna VARCHAR(100),
  IN p_cittaConsegna VARCHAR(50),
  IN p_capConsegna CHAR(5),
  IN p_referente VARCHAR(100),
  IN p_recapitoCorriere VARCHAR(50),
  IN p_dettagli JSON,
  OUT p_idOrdine INT
)
BEGIN
  DECLARE v_index INT DEFAULT 0;
  DECLARE v_count INT DEFAULT 0;
  DECLARE v_codiceSpecie CHAR(10);
  DECLARE v_quantita INT;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  IF p_codiceFornitore IS NULL OR p_codiceFornitore <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice fornitore non valido';
  END IF;

  IF NOT EXISTS (
    SELECT 1
    FROM Fornitore
    WHERE CodiceFornitore = p_codiceFornitore
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fornitore non trovato';
  END IF;

  IF p_dettagli IS NULL OR JSON_TYPE(p_dettagli) <> 'ARRAY' OR JSON_LENGTH(p_dettagli) =
0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Un ordine di rifornimento deve
contenere almeno un dettaglio';
  END IF;

  INSERT INTO OrdineRifornimento (
    CodiceFornitore, DataOrdine, Stato, EmailContatto,
    ViaConsegna, CittaConsegna, CapConsegna, Referente, RecapitoCorriere
  ) VALUES (
    p_codiceFornitore, p_dataOrdine, 'APERTO', p_emailContatto,
    p_viaConsegna, p_cittaConsegna, p_capConsegna, p_referente, p_recapitoCorriere
  );

  SET p_idOrdine = LAST_INSERT_ID();
  SET v_count = JSON_LENGTH(p_dettagli);

  WHILE v_index < v_count DO
    SET v_codiceSpecie = JSON_UNQUOTE(JSON_EXTRACT(p_dettagli, CONCAT('$[', v_index,
    '].speciesCode')));

```

```

        SET v_quantita = CAST(JSON_UNQUOTE(JSON_EXTRACT(p_dettagli, CONCAT('$[', v_index,
        '].quantity'))) AS UNSIGNED);

        IF v_codiceSpecie IS NULL OR CHAR_LENGTH(TRIM(v_codiceSpecie)) = 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice specie non valido';
        END IF;

        IF v_quantita IS NULL OR v_quantita <= 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La quantità deve essere maggiore
di zero';
        END IF;

        IF NOT EXISTS (
            SELECT 1
            FROM FornituraSpecie
            WHERE CodiceFornitore = p_codiceFornitore
               AND CodiceSpecie = v_codiceSpecie
               AND Attiva = TRUE
        ) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Il fornitore non può fornire una
delle specie richieste';
        END IF;

        INSERT INTO DettaglioOrdineRifornimento (IdOrdine, CodiceSpecie, Quantita)
        VALUES (p_idOrdine, v_codiceSpecie, v_quantita);

        SET v_index = v_index + 1;
    END WHILE;

    COMMIT;
END

```

La procedura **InserisciOrdineRifornimentoCompleto** gestisce atomicamente la creazione di un ordine di rifornimento e delle relative righe di dettaglio. Riceve i dati generali dell'ordine e un array JSON contenente le specie da ordinare e le rispettive quantità.

Prima dell'inserimento vengono verificati la validità del codice del fornitore, l'esistenza del fornitore e la presenza di almeno una riga di dettaglio. L'ordine viene creato nello stato iniziale APERTO.

Successivamente vengono elaborati i dettagli dell'ordine. Per ciascuna riga vengono verificati il codice della specie e la quantità richiesta. Viene inoltre controllato che il fornitore selezionato sia effettivamente abilitato a fornire la specie attraverso la relazione FornituraSpecie.

Se una specie non è valida, la quantità non è positiva oppure il fornitore non è abilitato a fornire la specie richiesta, la procedura interrompe l'operazione mediante SIGNAL SQLSTATE '45000'.

La creazione dell'ordine e di tutti i relativi dettagli viene eseguita all'interno della stessa transazione. In caso di errore viene effettuato il ROLLBACK dell'intera operazione e l'errore originario viene propagato mediante RESIGNAL, evitando la registrazione di ordini di rifornimento incompleti.

**Procedura RicercaOrdineRifornimento**

```
DELIMITER //
CREATE PROCEDURE `RicercaOrdineRifornimento` (
    IN p_idOrdine INT
)
BEGIN
SELECT
    o.IdOrdine,
    o.CodiceFornitore,
    f.Nome AS NomeFornitore,
    f.CodiceFiscale,
    o.DataOrdine,
    o.Stato,
    o.EmailContatto,
    o.ViaConsegna,
    o.CittaConsegna,
    o.CapConsegna,
    o.Referente,
    o.RecapitoCorriere,
    o.DataCreazione,
    o.DataModifica
FROM
    OrdineRifornimento o
JOIN
    Fornitore f ON o.CodiceFornitore = f.CodiceFornitore
WHERE
    o.IdOrdine = p_idOrdine;

SELECT
    d.CodiceSpecie,
    s.NomeComune,
    s.NomeLatino,
    d.Quantita
FROM
    DettaglioOrdineRifornimento d
JOIN
    SpecieDiPiante s ON d.CodiceSpecie = s.CodiceSpecie
WHERE
    d.IdOrdine = p_idOrdine
ORDER BY
    s.NomeComune;
END //
DELIMITER ;
```

Questa procedura permette di visualizzare una scheda completa dell'ordine, riportando sia i dati generali, sia i dettagli sulle specie e sulle quantità richieste. I dati sono ottenuti tramite join tra le tabelle OrdineRifornimento, Fornitore, DettaglioOrdineRifornimento e SpecieDiPiante.

**Procedura ModificaOrdineRifornimentoAperto**

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ModificaOrdineRifornimentoAperto`(  
    IN p_idOrdine INT,  
    IN p_emailContatto VARCHAR(100),  
    IN p_viaConsegna VARCHAR(100),  
    IN p_cittaConsegna VARCHAR(50),  
    IN p_capConsegna CHAR(5),  
    IN p_referente VARCHAR(100),  
    IN p_recapitoCorriere VARCHAR(50)  
)  
BEGIN  
    DECLARE v_stato VARCHAR(20);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    SET v_stato = NULL;  
    SELECT Stato INTO v_stato  
    FROM OrdineRifornimento  
    WHERE IdOrdine = p_idOrdine  
    FOR UPDATE;  
  
    IF v_stato IS NULL THEN  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ordine di rifornimento non trovato';  
    END IF;  
  
    IF v_stato <> 'APERTO' THEN  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Si possono modificare solamente ordini  
di rifornimento aperti';  
    END IF;  
  
    UPDATE OrdineRifornimento  
    SET EmailContatto = p_emailContatto,  
        ViaConsegna = p_viaConsegna,  
        CittaConsegna = p_cittaConsegna,  
        CapConsegna = p_capConsegna,  
        Referente = p_referente,  
        RecapitoCorriere = p_recapitoCorriere  
    WHERE IdOrdine = p_idOrdine;  
  
    COMMIT;  
END
```

La procedura **ModificaOrdineRifornimentoAperto** consente di modificare esclusivamente le informazioni di consegna e di contatto di un ordine di rifornimento che si trova ancora nello stato APERTO.

La procedura recupera l'ordine tramite il relativo identificativo e ne acquisisce la riga mediante FOR UPDATE, in modo da evitare modifiche concorrenti durante l'operazione.

Se l'ordine non esiste oppure si trova in uno stato diverso da APERTO, la procedura interrompe l'operazione mediante SIGNAL SQLSTATE '45000'.

Se i controlli hanno esito positivo, vengono aggiornati esclusivamente i dati relativi alla consegna e al contatto, come l'indirizzo di consegna, l'e-mail di contatto, il referente e il recapito del corriere. Non vengono invece modificati il fornitore, la data dell'ordine, lo stato e le righe di dettaglio.

L'operazione viene eseguita all'interno di una transazione. In caso di errore viene eseguito il ROLLBACK e l'errore originario viene propagato mediante RESIGNAL.

### Procedura EliminaOrdineRifornimento

```
CREATE DEFINER='root'@'localhost' PROCEDURE `EliminaOrdineRifornimento`(  
    IN p_idOrdine INT  
)  
BEGIN  
    DECLARE v_stato VARCHAR(20);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    SET v_stato = NULL;  
  
    SELECT Stato INTO v_stato  
    FROM OrdineRifornimento  
    WHERE IdOrdine = p_idOrdine  
    FOR UPDATE;  
  
    IF v_stato IS NULL THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Ordine di rifornimento non trovato';  
    END IF;  
  
    IF v_stato <> 'APERTO' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Si possono eliminare solamente ordini aperti!';  
    END IF;  
  
    DELETE FROM OrdineRifornimento  
    WHERE IdOrdine = p_idOrdine;  
  
    COMMIT;  
END
```

La procedura consente di eliminare un ordine di rifornimento esclusivamente se si trova nello stato APERTO. La riga dell'ordine viene acquisita mediante FOR UPDATE; se l'ordine non esiste viene generato un errore specifico, mentre se si trova in uno stato diverso da APERTO l'eliminazione viene impedita. L'operazione è eseguita in una transazione con isolamento READ COMMITTED; in caso di errore viene effettuato il ROLLBACK e l'errore originario viene propagato mediante RESIGNAL.

## Procedura VisualizzaOrdiniRifornimento

```
CREATE PROCEDURE VisualizzaOrdiniRifornimento()
BEGIN
    SELECT IdOrdine, CodiceFornitore, DataOrdine, Stato, EmailContatto,
           ViaConsegna, CittaConsegna, CapConsegna,
           Referente, RecapitoCorriere
    FROM OrdineRifornimento
    ORDER BY DataOrdine DESC, IdOrdine DESC;
END;
```

La procedura **VisualizzaOrdiniRifornimento** restituisce l'elenco completo degli ordini di rifornimento registrati nel sistema, mostrando i dati del fornitore, lo stato dell'ordine e le informazioni di contatto e consegna. I risultati sono ordinati dal più recente al meno recente.

## Procedura InserisciOrdineVenditaCompleto

```
CREATE DEFINER='root'@'localhost' PROCEDURE `InserisciOrdineVenditaCompleto`(
    IN p_partitaIVA CHAR(11),
    IN p_dataOrdine DATE,
    IN p_viaConsegna VARCHAR(100),
    IN p_cittaConsegna VARCHAR(50),
    IN p_capConsegna CHAR(5),
    IN p_referente VARCHAR(100),
    IN p_recapitoCorriere VARCHAR(50),
    IN p_dettagli JSON,
    OUT p_idOrdine INT
)
BEGIN
    DECLARE v_index INT DEFAULT 0;
    DECLARE v_count INT DEFAULT 0;
    DECLARE v_codiceSpecie CHAR(10);
    DECLARE v_quantita INT;
    DECLARE v_prezzoUnitario DECIMAL(10,2);
    DECLARE v_giacenza INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;

    IF p_partitaIVA IS NULL OR CHAR_LENGTH(TRIM(p_partitaIVA)) <> 11 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Partita IVA non valida';
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM AziendeRivenditrici
        WHERE PartitaIVA = p_partitaIVA
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Azienda non trovata';
    END IF;

    IF p_dettagli IS NULL OR JSON_TYPE(p_dettagli) <> 'ARRAY' OR JSON_LENGTH(p_dettagli) =
0 THEN
```

```

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Un ordine di vendita deve contenere
almeno un dettaglio';
    END IF;

    INSERT INTO OrdiniDiVendita (
        PartitaIVA, DataOrdine, Stato, ViaConsegna,
        CittaConsegna, CAPConsegna, Referente, RecapitoCorriere
    ) VALUES (
        p_partitaIVA, p_dataOrdine, 'APERTO', p_viaConsegna,
        p_cittaConsegna, p_capConsegna, p_referente, p_recapitoCorriere
    );

    SET p_idOrdine = LAST_INSERT_ID();
    SET v_count = JSON_LENGTH(p_dettagli);

    WHILE v_index < v_count DO
        SET v_codiceSpecie = JSON_UNQUOTE(JSON_EXTRACT(p_dettagli, CONCAT('$[', v_index,
        '].speciesCode')));
        SET v_quantita = CAST(JSON_UNQUOTE(JSON_EXTRACT(p_dettagli, CONCAT('$[', v_index,
        '].quantity'))) AS UNSIGNED);
        SET v_prezzoUnitario = CAST(JSON_UNQUOTE(JSON_EXTRACT(p_dettagli, CONCAT('$[',
        v_index, '].unitPrice'))) AS DECIMAL(10,2));

        IF v_codiceSpecie IS NULL OR CHAR_LENGTH(TRIM(v_codiceSpecie)) = 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Codice specie non valido';
        END IF;

        IF v_quantita IS NULL OR v_quantita <= 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La quantità deve essere maggiore
di zero';
        END IF;

        IF v_prezzoUnitario IS NULL OR v_prezzoUnitario <= 0 THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Il prezzo unitario deve essere
maggiore di zero';
        END IF;

        SET v_giacenza = NULL;
        SELECT Quantita INTO v_giacenza
        FROM Magazzino
        WHERE CodiceSpecie = v_codiceSpecie
        FOR UPDATE;

        IF v_giacenza IS NULL THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Specie non presente in magazzino';
        END IF;

        IF v_giacenza < v_quantita THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Giacenza insufficiente per
soddisfare l ordine';
        END IF;

        INSERT INTO DettagliOrdine (IdOrdine, CodiceSpecie, Quantita, PrezzoUnitario)
        VALUES (p_idOrdine, v_codiceSpecie, v_quantita, v_prezzoUnitario);

        SET v_index = v_index + 1;
    END WHILE;

    COMMIT;
END

```

La procedura **InserisciOrdineVenditaCompleto** gestisce atomicamente la creazione di un ordine di vendita e delle relative righe di dettaglio. Riceve i dati generali dell'ordine e un array JSON contenente le specie ordinate, le quantità e i prezzi unitari.

Prima dell'inserimento viene verificata l'esistenza dell'azienda rivenditrice e la validità dei dati richiesti. L'ordine viene creato nello stato iniziale APERTO e il relativo identificativo viene recuperato mediante `LAST_INSERT_ID()`.

Successivamente vengono elaborati i dettagli dell'ordine. Per ciascuna specie vengono verificati il codice, la quantità richiesta e la disponibilità in magazzino. La riga di magazzino viene acquisita mediante `FOR UPDATE`, in modo da garantire la consistenza della giacenza anche in presenza di operazioni concorrenti.

Se una specie non è presente in magazzino oppure la quantità disponibile è insufficiente, la procedura interrompe l'operazione mediante `SIGNAL SQLSTATE '45000'`.

Il prezzo unitario viene memorizzato direttamente nella riga di dettaglio dell'ordine. In questo modo il prezzo rimane invariato per l'intero ciclo di vita dell'ordine e non viene modificato dalle successive variazioni registrate nello storico dei prezzi.

La creazione dell'ordine e di tutti i relativi dettagli appartiene a un'unica transazione. In caso di errore viene eseguito il `ROLLBACK` dell'intera operazione e l'errore originario viene propagato mediante `RESIGNAL`, evitando la registrazione di ordini incompleti.

**Procedura ModificaOrdineVenditaAperto**

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `ModificaOrdineVenditaAperto`(
  IN p_idOrdine INT,
  IN p_viaConsegna VARCHAR(100),
  IN p_cittaConsegna VARCHAR(50),
  IN p_capConsegna CHAR(5),
  IN p_referente VARCHAR(100),
  IN p_recapitoCorriere VARCHAR(50)
)
BEGIN
  DECLARE v_stato VARCHAR(20);

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  SET v_stato = NULL;
  SELECT Stato INTO v_stato
  FROM OrdiniDiVendita
  WHERE IdOrdine = p_idOrdine
  FOR UPDATE;

  IF v_stato IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ordine non trovato';
  END IF;

  IF v_stato <> 'APERTO' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Si possono modificare solamente ordini
aperti';
  END IF;

  UPDATE OrdiniDiVendita
  SET ViaConsegna = p_viaConsegna,
      CittaConsegna = p_cittaConsegna,
      CAPConsegna = p_capConsegna,
      Referente = p_referente,
      RecapitoCorriere = p_recapitoCorriere
  WHERE IdOrdine = p_idOrdine;

  COMMIT;
END

```

La procedura **ModificaOrdineVenditaAperto** consente di modificare esclusivamente le informazioni di consegna di un ordine di vendita che si trova ancora nello stato APERTO.

La procedura recupera l'ordine tramite il relativo identificativo e ne acquisisce la riga mediante FOR UPDATE, in modo da evitare modifiche concorrenti durante l'operazione.

Se l'ordine non esiste oppure si trova in uno stato diverso da APERTO, la procedura interrompe l'operazione mediante SIGNAL SQLSTATE '45000'.

Se i controlli hanno esito positivo, vengono aggiornati esclusivamente l'indirizzo di consegna, il referente e il recapito del corriere. Non vengono invece modificati l'azienda cliente, la data dell'ordine, lo stato e le righe di dettaglio.

L'operazione viene eseguita all'interno di una transazione. In caso di errore viene eseguito il ROLLBACK e l'errore originario viene propagato mediante RESIGNAL.

### Procedura VisualizzaOrdiniVendita

```
CREATE PROCEDURE VisualizzaOrdiniVendita()  
BEGIN  
    SELECT IdOrdine, PartitaIVA, DataOrdine, Stato,  
           ViaConsegna, CittaConsegna, CAPConsegna,  
           Referente, RecapitoCorriere  
    FROM OrdiniDiVendita  
    ORDER BY DataOrdine DESC, IdOrdine DESC;  
END;
```

La procedura **VisualizzaOrdiniVendita** restituisce l'elenco completo degli ordini di vendita registrati nel sistema, mostrando l'azienda cliente, la data, lo stato e le informazioni di consegna. I risultati vengono ordinati dal più recente al meno recente.

### Procedura EliminaOrdineVendita

```
CREATE DEFINER='root'@'localhost' PROCEDURE `EliminaOrdineVendita`(IN p_idOrdine INT)  
BEGIN  
    DECLARE v_stato VARCHAR(20);  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    SELECT Stato INTO v_stato  
    FROM OrdiniDiVendita  
    WHERE IdOrdine = p_idOrdine  
    FOR UPDATE;  
  
    IF v_stato IS NULL THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Ordine non trovato';  
    END IF;  
  
    IF v_stato <> 'APERTO' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Si possono eliminare solamente ordini aperti!';  
    END IF;  
  
    DELETE FROM OrdiniDiVendita  
    WHERE IdOrdine = p_idOrdine;  
  
    COMMIT;  
END
```

Questa procedura consente di eliminare un ordine di vendita solo se si trova nello stato APERTO. Avvia una transazione con livello di isolamento READ COMMITTED, blocca la riga dell'ordine con FOR UPDATE e ne verifica l'esistenza. Se l'ordine non esiste o se lo stato è diverso da APERTO, viene generato un errore tramite SIGNAL. In caso positivo, l'ordine viene eliminato dalla tabella principale OrdiniDiVendita e grazie al vincolo ON DELETE CASCADE, anche i relativi dettagli vengono rimossi automaticamente.

### Procedura VisualizzaDettagliOrdineVendita

```
CREATE PROCEDURE VisualizzaDettagliOrdineVendita(  
    IN p_idOrdine INT  
)  
BEGIN  
    SELECT IdOrdine, CodiceSpecie, Quantita, PrezzoUnitario  
    FROM DettagliOrdine  
    WHERE IdOrdine = p_idOrdine  
    ORDER BY CodiceSpecie;  
END;
```

La procedura **VisualizzaDettagliOrdineVendita** restituisce le righe di dettaglio associate a uno specifico ordine di vendita, mostrando per ciascuna specie la quantità ordinata e il prezzo unitario memorizzato al momento dell'ordine.

### Procedura RicercaOrdineVendita

```
CREATE PROCEDURE RicercaOrdineVendita(  
    IN p_idOrdine INT  
)  
BEGIN  
    SELECT IdOrdine, PartitaIVA, DataOrdine, Stato,  
           ViaConsegna, CittaConsegna, CAPConsegna,  
           Referente, RecapitoCorriere  
    FROM OrdiniDiVendita  
    WHERE IdOrdine = p_idOrdine;  
END;
```

La procedura **RicercaOrdineVendita** recupera i dati generali di uno specifico ordine di vendita tramite il relativo identificativo, restituendo l'azienda cliente, la data e lo stato dell'ordine e le informazioni necessarie alla consegna.

**Procedura ModificaStatoOrdine**

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ModificaStatoOrdine` (
  IN p_idOrdine INT,
  IN p_nuovoStato ENUM(
    'APERTO',
    'CONFERMATO',
    'SPEDITO',
    'CONSEGNATO',
    'ANNULLATO'
  )
)
BEGIN
  DECLARE stato_corrente ENUM(
    'APERTO',
    'CONFERMATO',
    'SPEDITO',
    'CONSEGNATO',
    'ANNULLATO'
  );

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  SELECT Stato INTO stato_corrente
  FROM OrdiniDiVendita
  WHERE IdOrdine = p_idOrdine
  FOR UPDATE;

  IF stato_corrente IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Ordine non trovato';
  END IF;

  UPDATE OrdiniDiVendita
  SET Stato = p_nuovoStato
  WHERE IdOrdine = p_idOrdine;

  COMMIT;
END
```

La procedura `ModificaStatoOrdine` aggiorna lo stato di un ordine di vendita. Prima dell'aggiornamento verifica l'esistenza dell'ordine e ne blocca la riga mediante `FOR UPDATE`. L'operazione viene eseguita in una transazione con isolamento `READ COMMITTED`; in caso di errore viene effettuato il `ROLLBACK` e l'errore originario viene propagato mediante `RESIGNAL`. La validità delle transizioni di stato è inoltre verificata dai trigger associati alla tabella degli ordini di vendita.

**Procedura OttieniQuantitaGiacenza**

```
CREATE PROCEDURE OttieniQuantitaGiacenza(  
    IN p_CodiceSpecie CHAR(10)  
)  
BEGIN  
    SELECT COALESCE((  
        SELECT Quantita  
        FROM Magazzino  
        WHERE CodiceSpecie = p_CodiceSpecie  
        LIMIT 1  
    ), 0) AS Quantita;  
END;
```

La procedura **OttieniQuantitaGiacenza** restituisce la quantità attualmente disponibile in magazzino per una determinata specie. Se per la specie indicata non è presente alcuna riga nella tabella Magazzino, viene restituito il valore 0.

**Procedura ModificaStatoOrdineRifornimento**

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ModificaStatoOrdineRifornimento`(  
    IN p_idOrdine INT,  
    IN p_nuovoStato ENUM(  
        'APERTO', 'CONFERMATO', 'SPEDITO', 'CONSEGNATO', 'ANNULLATO'  
    )  
)  
BEGIN  
    DECLARE order_exists INT DEFAULT 0;  
    DECLARE msg TEXT;  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
    SELECT COUNT(*) INTO order_exists  
    FROM OrdineRifornimento  
    WHERE IdOrdine = p_idOrdine  
    FOR UPDATE;  
    IF order_exists = 0 THEN  
        SET msg = CONCAT(  
            'Ordine di rifornimento con ID ',  
            p_idOrdine,  
            ' non trovato'  
        );  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = msg;  
    END IF;  
    UPDATE OrdineRifornimento  
    SET Stato = p_nuovoStato  
    WHERE IdOrdine = p_idOrdine;  
    COMMIT;  
END
```

La procedura **ModificaStatoOrdineRifornimento** modifica lo stato di un ordine di rifornimento. Prima dell'aggiornamento verifica l'esistenza dell'ordine e ne acquisisce la riga mediante FOR UPDATE. Se l'ordine non esiste viene generato un errore specifico.

### Procedura VisualizzaDettagliOrdineRifornimento

```
CREATE PROCEDURE VisualizzaDettagliOrdineRifornimento(  
    IN p_idOrdine INT  
)  
BEGIN  
    SELECT IdOrdine, CodiceSpecie, Quantita  
    FROM DettaglioOrdineRifornimento  
    WHERE IdOrdine = p_idOrdine  
    ORDER BY CodiceSpecie;  
END;
```

La procedura **VisualizzaDettagliOrdineRifornimento** restituisce le righe di dettaglio associate a uno specifico ordine di rifornimento, mostrando per ciascuna specie il relativo codice e la quantità richiesta.

### Procedura RicercaOrdiniPerStato

```
DELIMITER //  
CREATE PROCEDURE `RicercaOrdiniPerStato` (  
    IN p_stato ENUM('APERTO','CONFERMATO','SPEDITO','CONSEGNATO','ANNULLATO') )  
BEGIN  
    SELECT  
        o.IdOrdine,  
        o.PartitaIVA,  
        a.NomeAzienda,  
        o.DataOrdine,  
        o.Stato,  
        o.ViaConsegna,  
        o.CittaConsegna,  
        o.CAPConsegna,  
        o.Referente,  
        o.RecapitoCorriere,  
        (SELECT SUM(d.Quantita * d.PrezzoUnitario) FROM DettaglioOrdine d WHERE d.IdOrdine  
        = o.IdOrdine) AS ValoreTotale  
    FROM  
        OrdiniDiVendita o  
    JOIN  
        AziendeRivenditrici a ON o.PartitaIVA = a.PartitaIVA  
    WHERE  
        o.Stato = p_stato  
    ORDER BY  
        o.DataOrdine DESC;  
END //  
DELIMITER ;
```

Questa procedura restituisce tutti gli ordini che si trovano in un determinato stato, mostrando i dati generali e calcolando anche il valore complessivo tramite la somma dei dettagli.

## Procedura RicercaOrdiniPerAzienda

```

DELIMITER //
CREATE PROCEDURE `RicercaOrdiniPerAzienda`(
    IN p_partita_iva CHAR(11)
)
BEGIN
SELECT
    o.IdOrdine,
    o.PartitaIVA,
    a.NomeAzienda,
    o.DataOrdine,
    o.Stato,
    o.ViaConsegna,
    o.CittaConsegna,
    o.CAPConsegna,
    o.Referente,
    o.RecapitoCorriere,
    (SELECT SUM(d.Quantita * d.PrezzoUnitario) FROM DettagliOrdine d WHERE d.IdOrdine
= o.IdOrdine) AS ValoreTotale
    FROM
        OrdiniDiVendita o
    JOIN
        AziendeRivenditrici a ON o.PartitaIVA = a.PartitaIVA
    WHERE
        o.PartitaIVA = p_partita_iva
    ORDER BY
        o.DataOrdine DESC;
END //
DELIMITER ;

```

Questa procedura elenca tutti gli ordini effettuati da una specifica azienda.

## Procedura InserisciVariazionePrezzo

```

CREATE DEFINER=`root`@`localhost` PROCEDURE `InserisciVariazionePrezzo`(
    IN var_SpeciePianta CHAR(10),
    IN var_NuovoPrezzo DECIMAL(10,2)
)
BEGIN
    DECLARE var_PrezzoCorrente DECIMAL(10,2) DEFAULT NULL;
    DECLARE var_DataAttuale DATETIME(6);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
    START TRANSACTION;

    IF var_SpeciePianta IS NULL OR CHAR_LENGTH(TRIM(var_SpeciePianta)) = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Il codice della specie è obbligatorio.';
    END IF;

    IF NOT EXISTS (
        SELECT 1

```

```

        FROM SpecieDiPiante
        WHERE CodiceSpecie = var_SpeciePianta
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Specie non trovata.';
    END IF;

    IF var_NuovoPrezzo IS NULL OR var_NuovoPrezzo <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Il prezzo deve essere maggiore di zero.';
    END IF;

    SELECT PrezzoAttuale
    INTO var_PrezzoCorrente
    FROM StoricoPrezzi
    WHERE SpeciePianta = var_SpeciePianta
    ORDER BY DataVariazione DESC
    LIMIT 1
    FOR UPDATE;

    IF var_PrezzoCorrente IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Nessun prezzo corrente trovato per la specie.';
    END IF;

    SET var_DataAttuale = NOW(6);

    INSERT INTO StoricoPrezzi (
        DataVariazione, PrezzoPrecedente, PrezzoAttuale, SpeciePianta
    ) VALUES (
        var_DataAttuale, var_PrezzoCorrente, var_NuovoPrezzo, var_SpeciePianta
    );

    COMMIT;
END

```

La procedura **InserisciVariazionePrezzo** registra una nuova variazione nello storico dei prezzi. Dopo aver verificato l'esistenza della specie e la validità del nuovo prezzo, recupera il prezzo attualmente in vigore selezionando la registrazione più recente dello storico.

La lettura viene effettuata all'interno della transazione e la riga corrispondente al prezzo corrente viene acquisita tramite FOR UPDATE. Successivamente viene inserita una nuova riga in StoricoPrezzi, impostando PrezzoPrecedente al prezzo precedentemente in vigore, PrezzoAttuale al nuovo valore e DataVariazione a NOW(6).

Ogni variazione produce sempre una nuova registrazione. Non viene utilizzato ON DUPLICATE KEY UPDATE, poiché una variazione successiva non deve sovrascrivere le precedenti. In caso di errore l'handler esegue il rollback della transazione e utilizza RESIGNAL per preservare il messaggio originario.

In presenza di un errore la transazione viene annullata e l'errore originale viene propagato tramite RESIGNAL.

### Procedura OttieniPrezzoAttuale

```
CREATE DEFINER='root'@'localhost' PROCEDURE `OttieniPrezzoAttuale`(  
  IN p_CodiceSpecie CHAR(10)  
)  
BEGIN  
  SELECT PrezzoAttuale  
  FROM StoricoPrezzi  
  WHERE SpeciePianta = p_CodiceSpecie  
  ORDER BY DataVariazione DESC  
  LIMIT 1;  
END
```

La procedura **OttieniPrezzoAttuale** determina il prezzo corrente di una specie selezionando la registrazione più recente presente in **StoricoPrezzi**, ordinata per **DataVariazione** in ordine decrescente.

In assenza di registrazioni la procedura non restituisce alcuna riga. Non viene utilizzato il valore zero come valore convenzionale per indicare l'assenza di un prezzo, poiché nel dominio applicativo un prezzo valido deve essere maggiore di zero.

### Procedura VisualizzaStoricoPrezzi

```
DELIMITER //  
CREATE PROCEDURE `VisualizzaStoricoPrezzi`(  
  IN codiceSpecie CHAR(10)  
)  
BEGIN  
  SELECT DataVariazione, PrezzoPrecedente, PrezzoAttuale  
  FROM StoricoPrezzi  
  WHERE SpeciePianta = codiceSpecie  
  ORDER BY DataVariazione DESC;  
END // DELIMITER ;
```

Questa procedura restituisce lo storico dei prezzi per una specie, con ordine dalla più recente alla meno recente.

## Procedura InserisciUtente

```
DELIMITER //
CREATE PROCEDURE `InserisciUtente` (
  IN var_Username VARCHAR(45),
  IN var_Password VARCHAR(64),
  IN var_Ruolo ENUM('amministratore', 'responsabileCommerciale', 'responsabileLogistico') )
BEGIN
  DECLARE EXIT HANDLER FOR 1062
  BEGIN
    ROLLBACK;
    RESIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Username già esistente.';
  END;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
  START TRANSACTION;

  INSERT INTO `Utenti` (`Username`, `Password`, `Ruolo`)
  VALUES (var_Username, SHA2(var_Password, 256), var_Ruolo);

  COMMIT;
END // DELIMITER ;
```

Questa procedura sarà utilizzata dall'amministratore per creare un nuovo utente con username, password e ruolo.

## Procedura VisualizzaUtenti

```
DELIMITER //
CREATE PROCEDURE `VisualizzaUtenti`()
BEGIN
  SELECT Username, Ruolo
  FROM Utenti
  ORDER BY Ruolo, Username;
END // DELIMITER ;
```

Questa procedura elenca gli utenti con il relativo ruolo.

**Procedura EliminaUtente**

```
DELIMITER //  
CREATE PROCEDURE `EliminaUtente`(  
    IN var_Username VARCHAR(45)  
)  
BEGIN  
    DECLARE v_rows INT DEFAULT 0;  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
    BEGIN  
        ROLLBACK;  
        RESIGNAL;  
    END;  
  
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
    START TRANSACTION;  
  
    DELETE FROM `Utenti` WHERE `Username` = var_Username;  
    SET v_rows = ROW_COUNT();  
  
    IF v_rows = 0 THEN  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Utente non trovato.';  
    END IF;  
  
    COMMIT;  
END // DELIMITER ;
```

Elimina un utente identificato dallo username.

**Procedura login**

```

CREATE PROCEDURE `login`(
    IN  var_Username VARCHAR(45),
    IN  var_pass VARCHAR(64),
    OUT var_role INT,
    OUT var_error_code INT,
    OUT var_error_message VARCHAR(100) )
BEGIN
    DECLARE var_user_role ENUM('amministratore', 'responsabileCommerciale',
    'responsabileLogistico');
    DECLARE var_user_exists INT DEFAULT 0;

    SET var_role = -1;
    SET var_error_code = NULL;
    SET var_error_message = NULL;

    SELECT COUNT(*) INTO var_user_exists
    FROM Utenti
    WHERE Username = var_Username;

    IF var_user_exists = 0 THEN
        SET var_role = -1;
        SET var_error_code = 1;
        SET var_error_message = 'Username non trovato';
    ELSE
        SELECT Ruolo INTO var_user_role
        FROM Utenti
        WHERE Username = var_Username
        AND Password = SHA2(var_pass, 256)
        LIMIT 1;

        IF var_user_role IS NULL THEN
            SET var_role = -1;
            SET var_error_code = 2;
            SET var_error_message = 'Password errata';
        ELSE
            IF var_user_role = 'amministratore' THEN
                SET var_role = 1;
            ELSEIF var_user_role = 'responsabileCommerciale' THEN
                SET var_role = 2;
            ELSEIF var_user_role = 'responsabileLogistico' THEN
                SET var_role = 3;
            ELSE
                SET var_role = -1;
                SET var_error_code = 3;
                SET var_error_message = 'Ruolo non valido';
            END IF;

            IF var_role > 0 THEN
                SET var_error_code = 0;
                SET var_error_message = 'Login riuscito';
            END IF;
        END IF;
    END IF;
END //

```

Questa procedura è utilizzata per effettuare l'autenticazione.

## Gestione degli errori tra layer applicativo e persistenza

Il client Java accede al database mediante il pattern DAO, mantenendo separata la logica applicativa dai dettagli della tecnologia JDBC.

Le stored procedure utilizzano SIGNAL SQLSTATE '45000' per segnalare le violazioni delle regole applicative. Quando si verifica un errore all'interno di una transazione, gli handler eseguono il ROLLBACK e utilizzano RESIGNAL per propagare l'errore originario senza sostituirne il messaggio.

Nel layer di persistenza le SQLException generate da JDBC vengono convertite in DaoException. La traduzione viene centralizzata mediante SQLExceptionTranslator: quando lo SQLSTATE è 45000, il messaggio applicativo proveniente dal database viene preservato, mentre per gli errori tecnici viene utilizzato un messaggio più generale.

I controller applicativi intercettano successivamente le DaoException e le incapsulano in ApplicationException, mantenendo l'eccezione originaria come causa. In questo modo gli strati superiori dell'applicazione non dipendono direttamente da JDBC e la View riceve errori espressi nel linguaggio del livello applicativo.